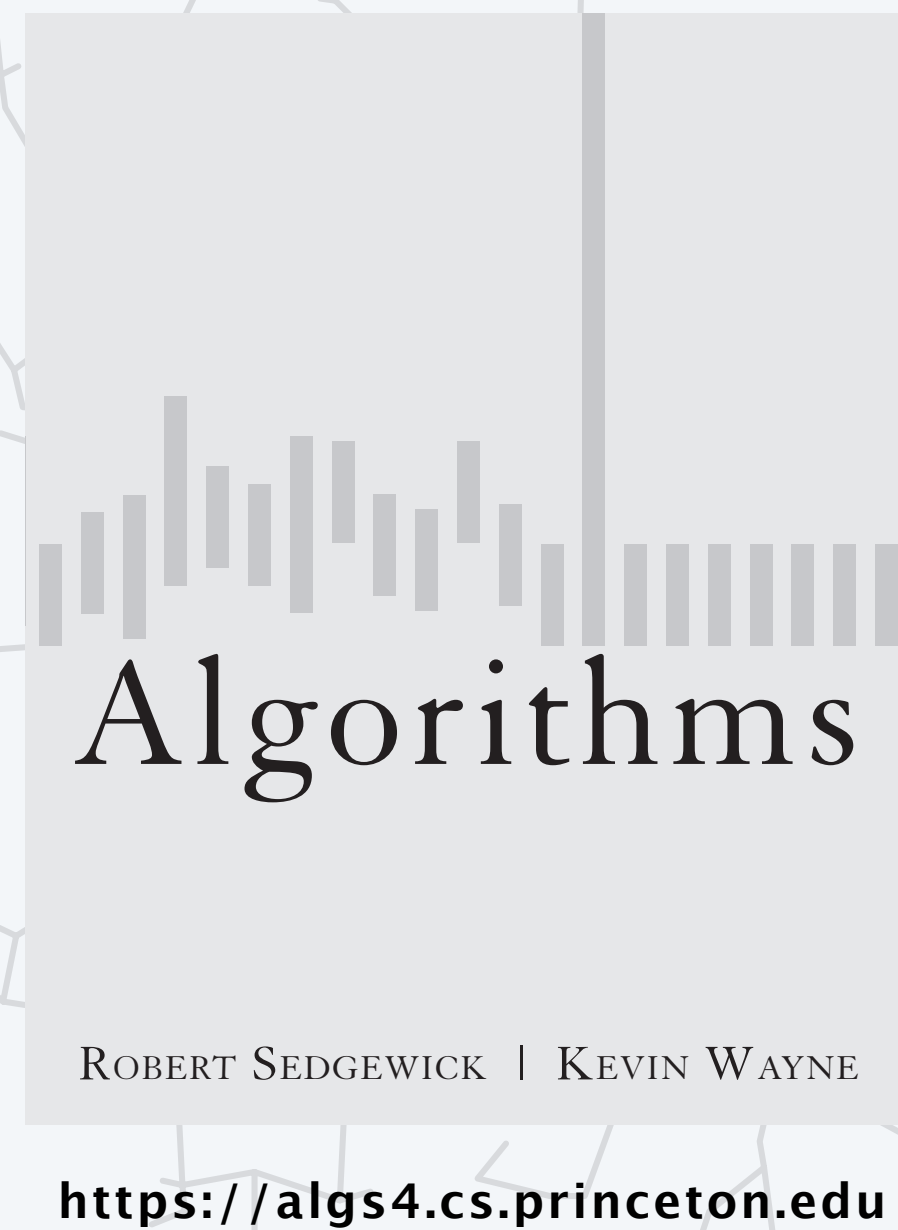




<https://algs4.cs.princeton.edu>

4.3 MINIMUM SPANNING TREES

- *introduction*
- *cut property*
- *edge-weighted graph API*
- *Kruskal's algorithm*
- *Prim's algorithm*

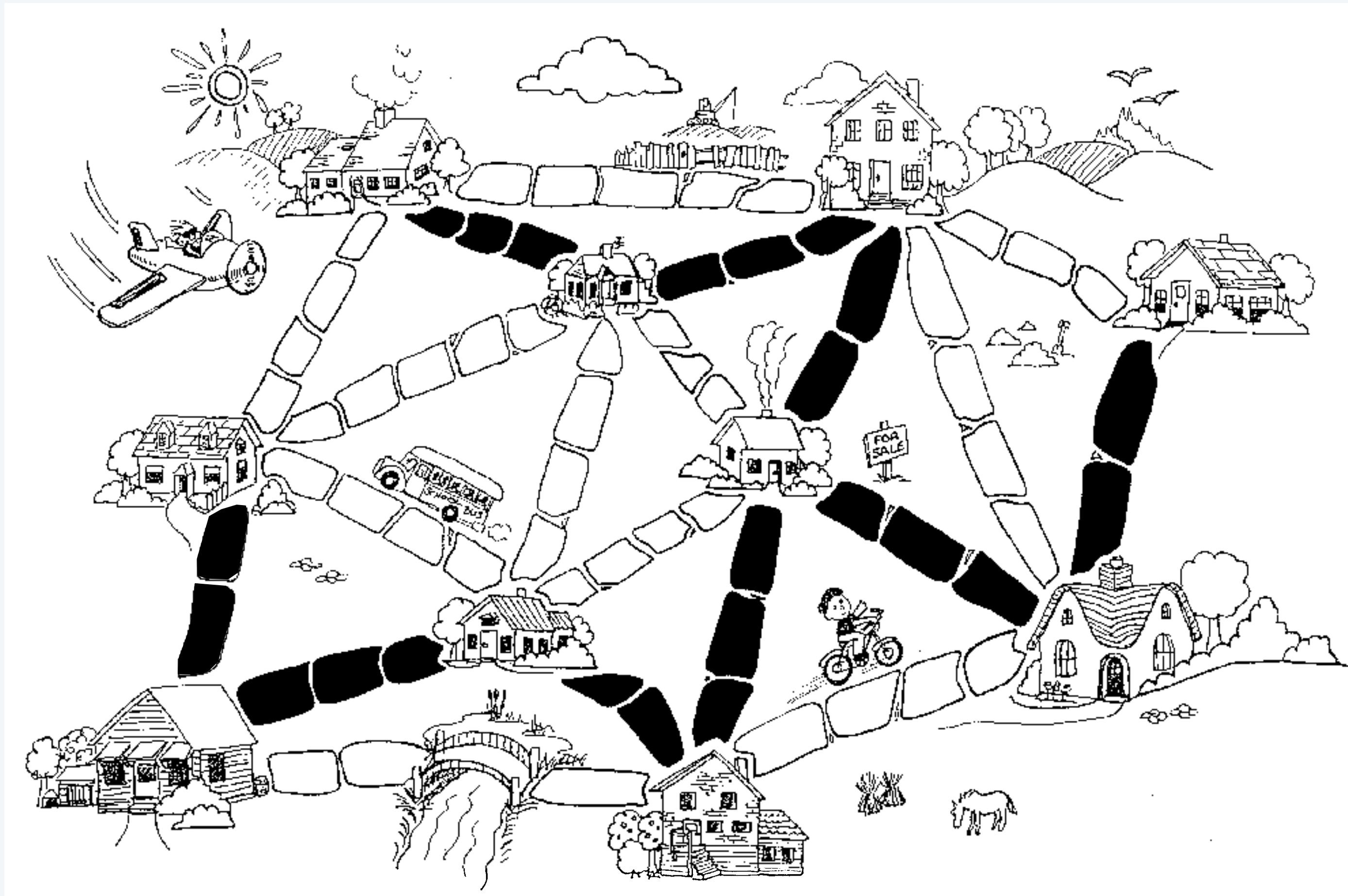


4.3 MINIMUM SPANNING TREES

- *introduction*
- *cut property*
- *edge-weighted graph API*
- *Kruskal's algorithm*
- *Prim's algorithm*

A motivating example

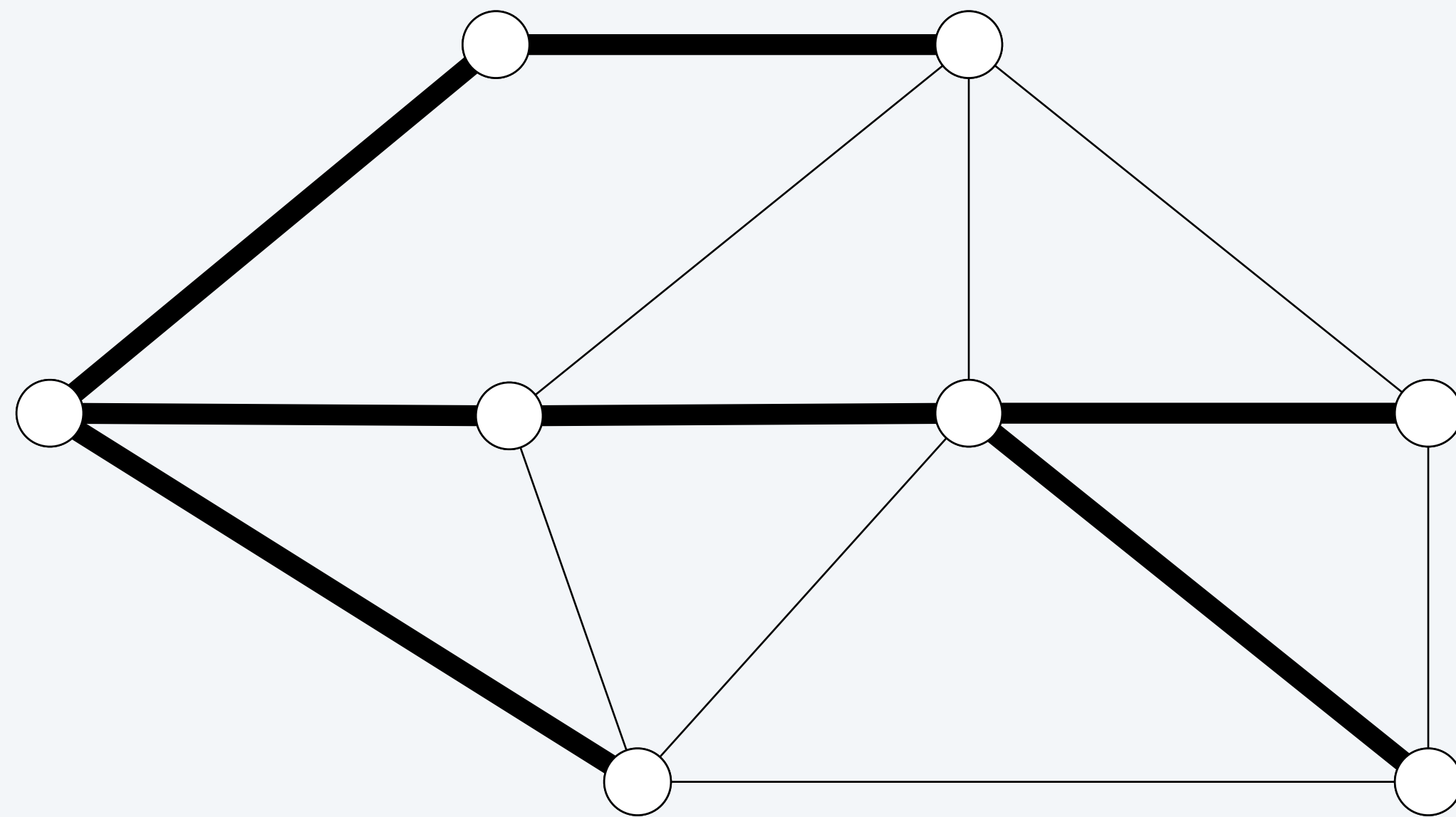
Install minimum number of paving stones to connect all of the houses.



Spanning tree

Def. A **spanning tree** of G is a subgraph T that is:

- A tree: connected and acyclic.
- Spanning: includes all of the vertices.



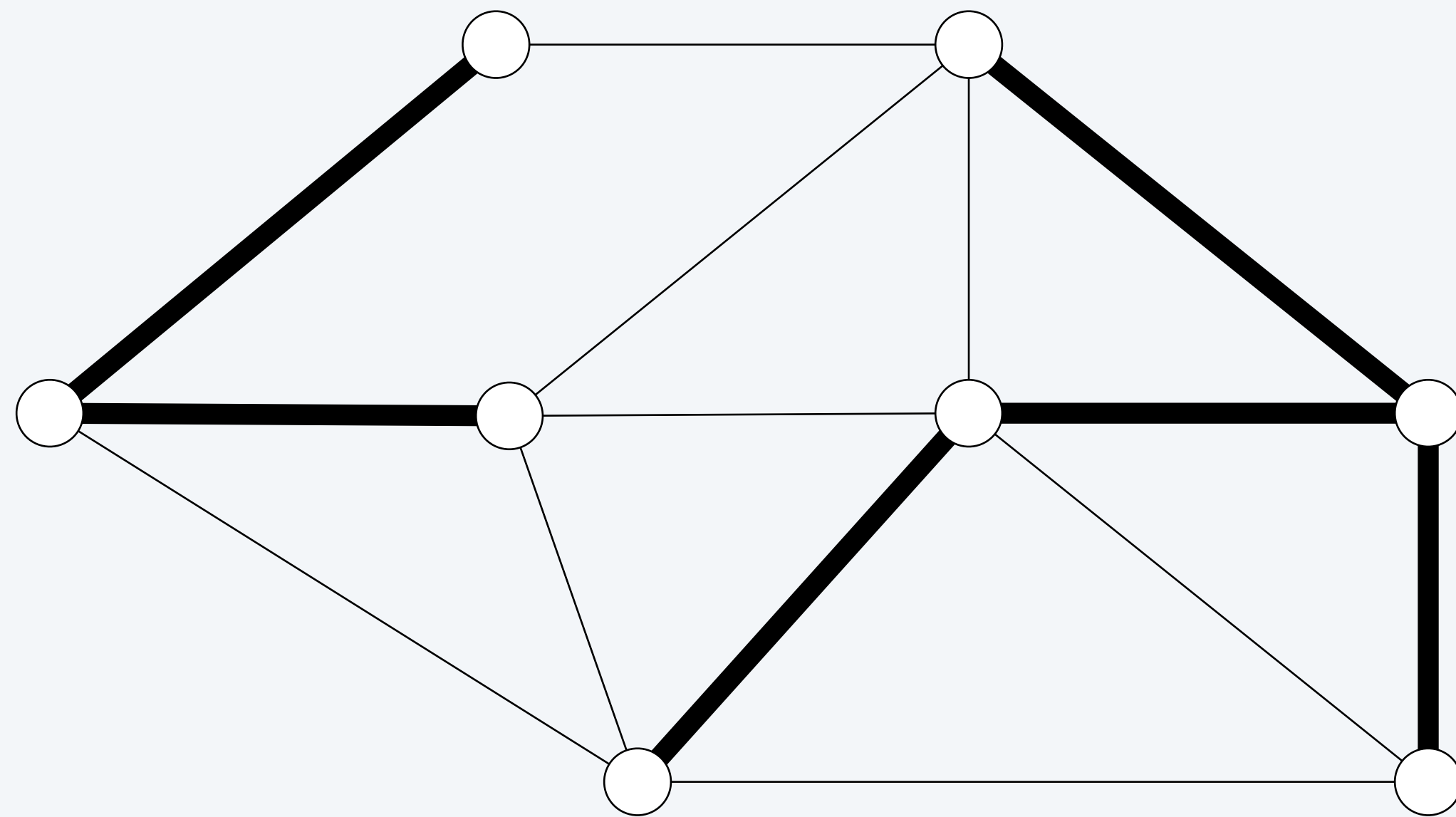
graph G

spanning tree T

Spanning tree

Def. A **spanning tree** of G is a subgraph T that is:

- A tree: connected and acyclic.
- Spanning: includes all of the vertices.

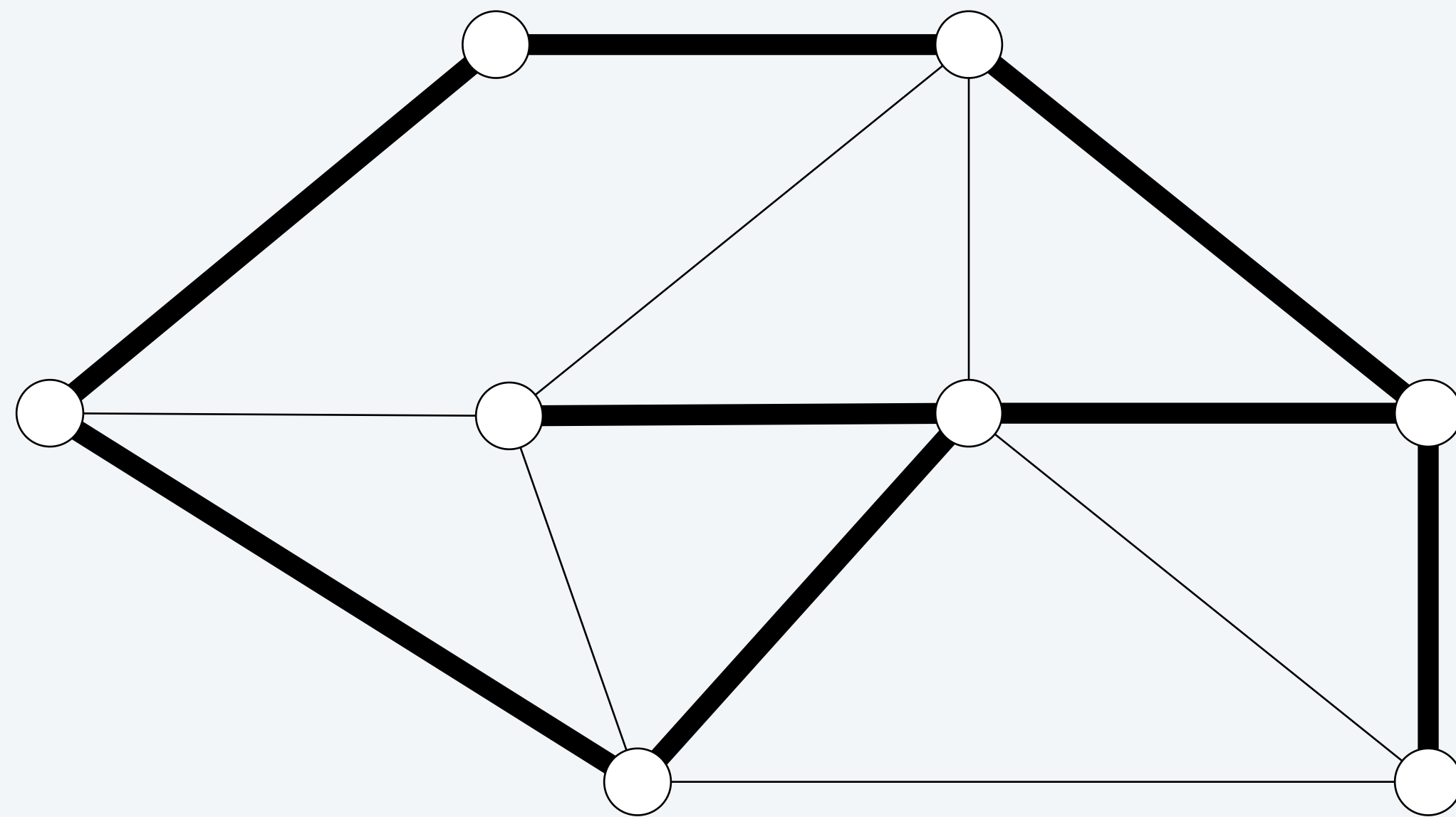


not connected

Spanning tree

Def. A **spanning tree** of G is a subgraph T that is:

- A tree: connected and acyclic.
- Spanning: includes all of the vertices.

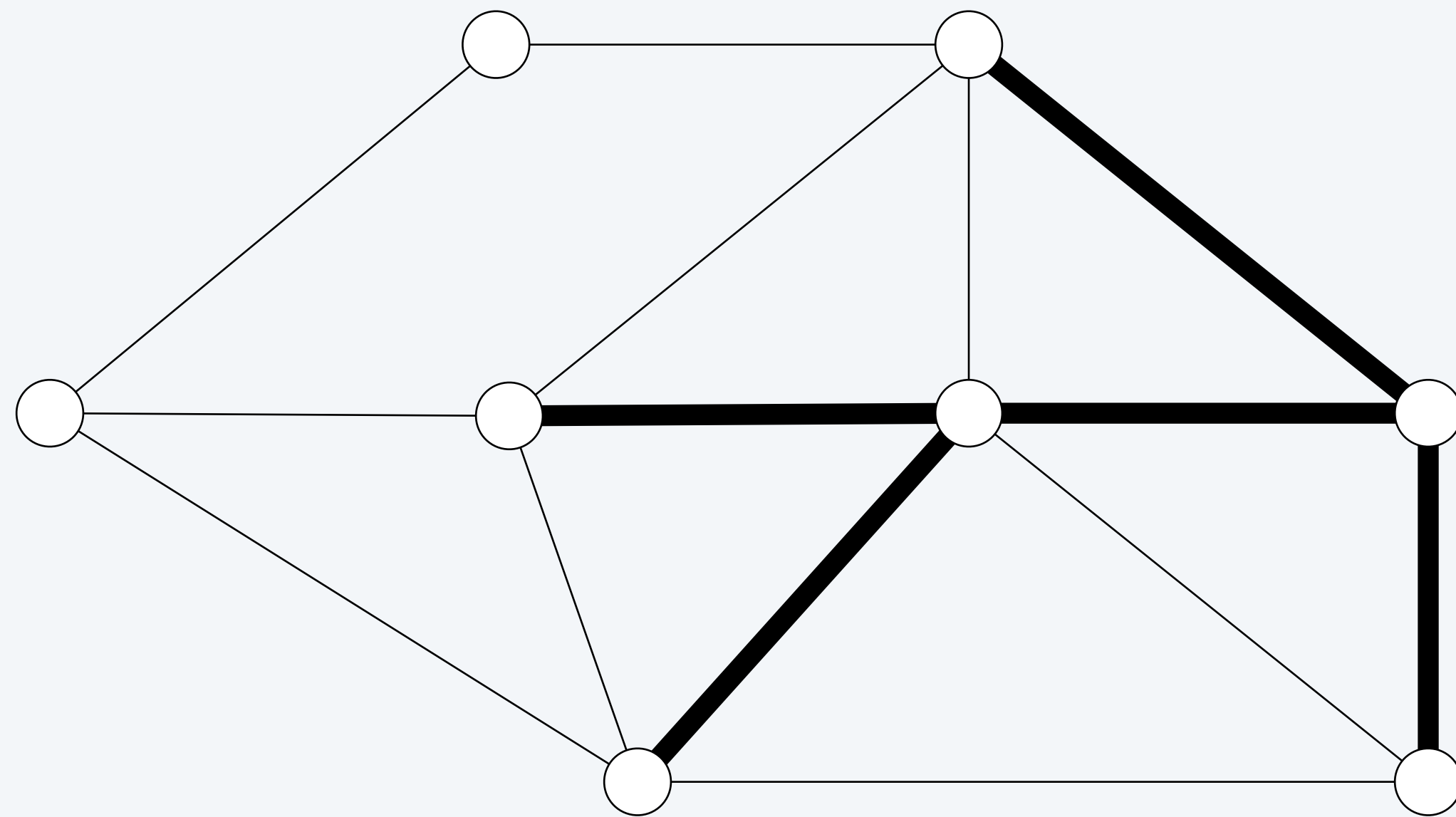


not acyclic

Spanning tree 生成树

Def. A **spanning tree** of G is a subgraph T that is:

- A tree: connected and acyclic. 连接 && 无环
- Spanning: includes all of the vertices. 包含所有节点

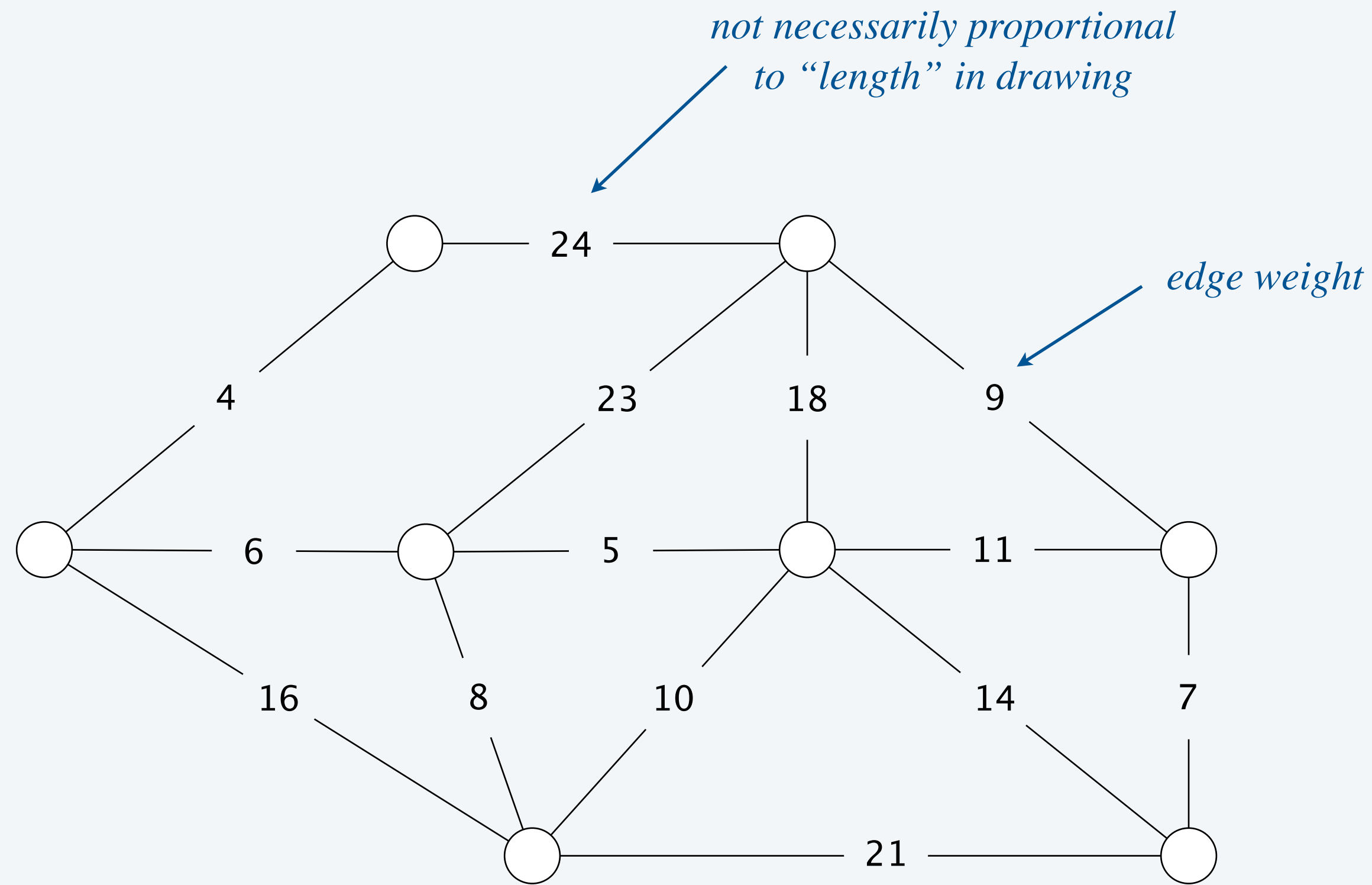


not spanning

Minimum spanning tree problem

最小生成树

Input. Connected, undirected graph G with positive edge weights.

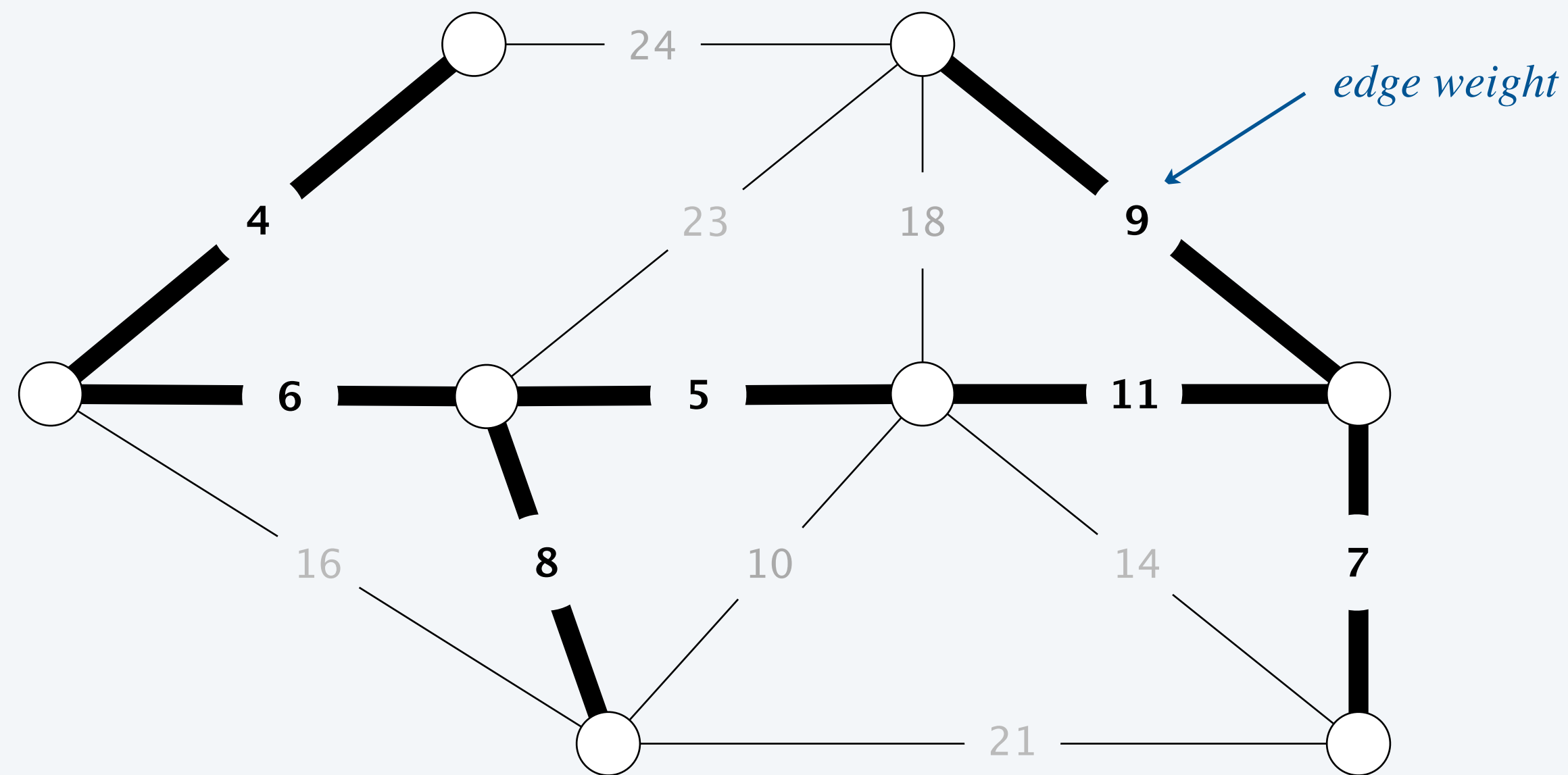


edge-weighted graph G

Minimum spanning tree problem

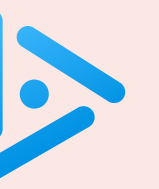
Input. Connected, undirected graph G with positive edge weights.

Output. A spanning tree of minimum weight.



minimum spanning tree T
(weight = 50 = 4 + 6 + 5 + 8 + 9 + 11 + 7)

Brute force. Try all spanning trees?



Let T be any spanning tree of a connected graph G with V vertices.

Which of the following properties must hold?

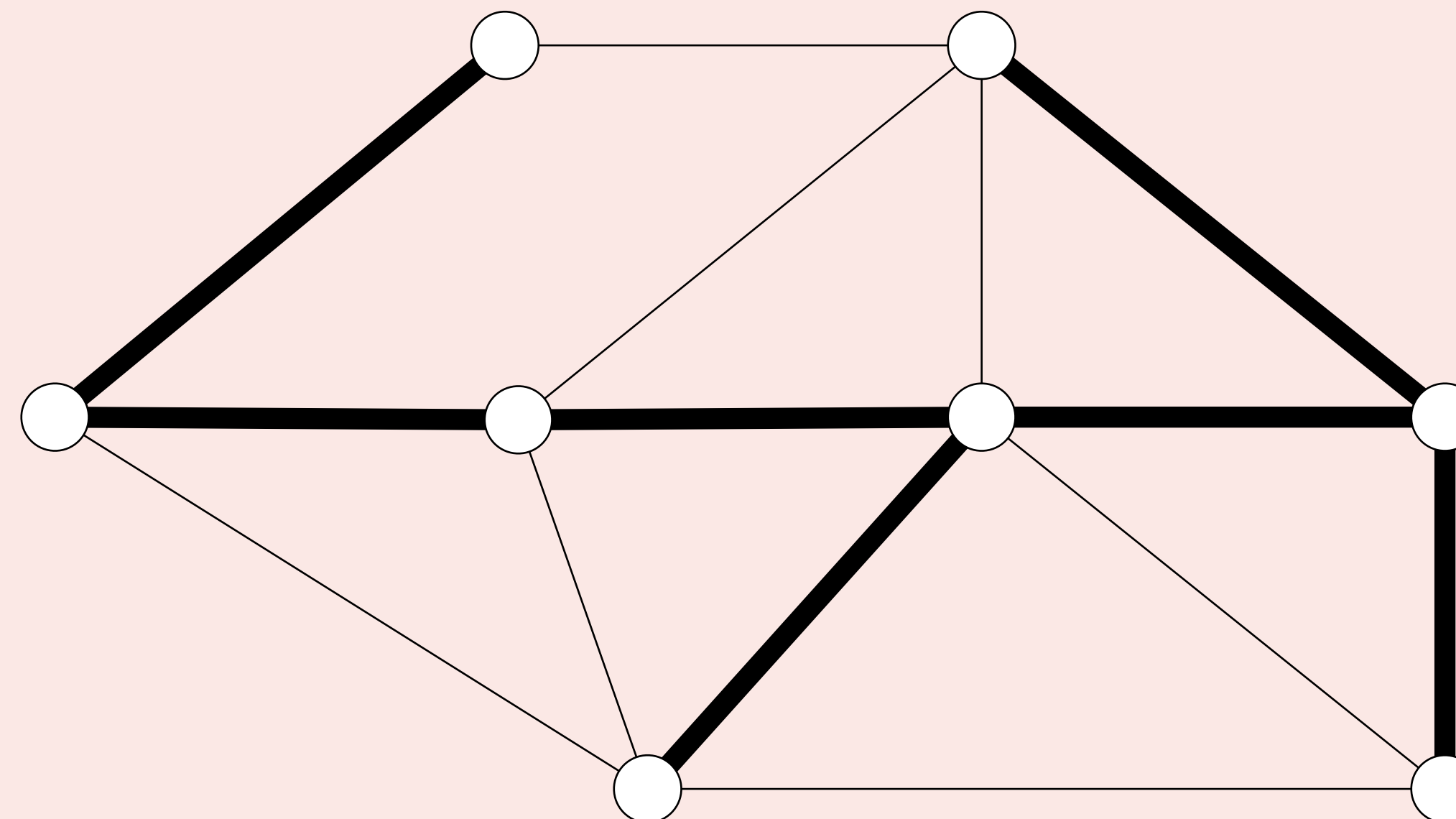
所有的性质都可以围绕“生成树没有环”+“所有的节点相互连通的”来思考

A. Removing any edge from T disconnects it.

B. Adding any edge to T creates a cycle.

C. T contains exactly $V - 1$ edges. 不多也不少！

D. All of the above.

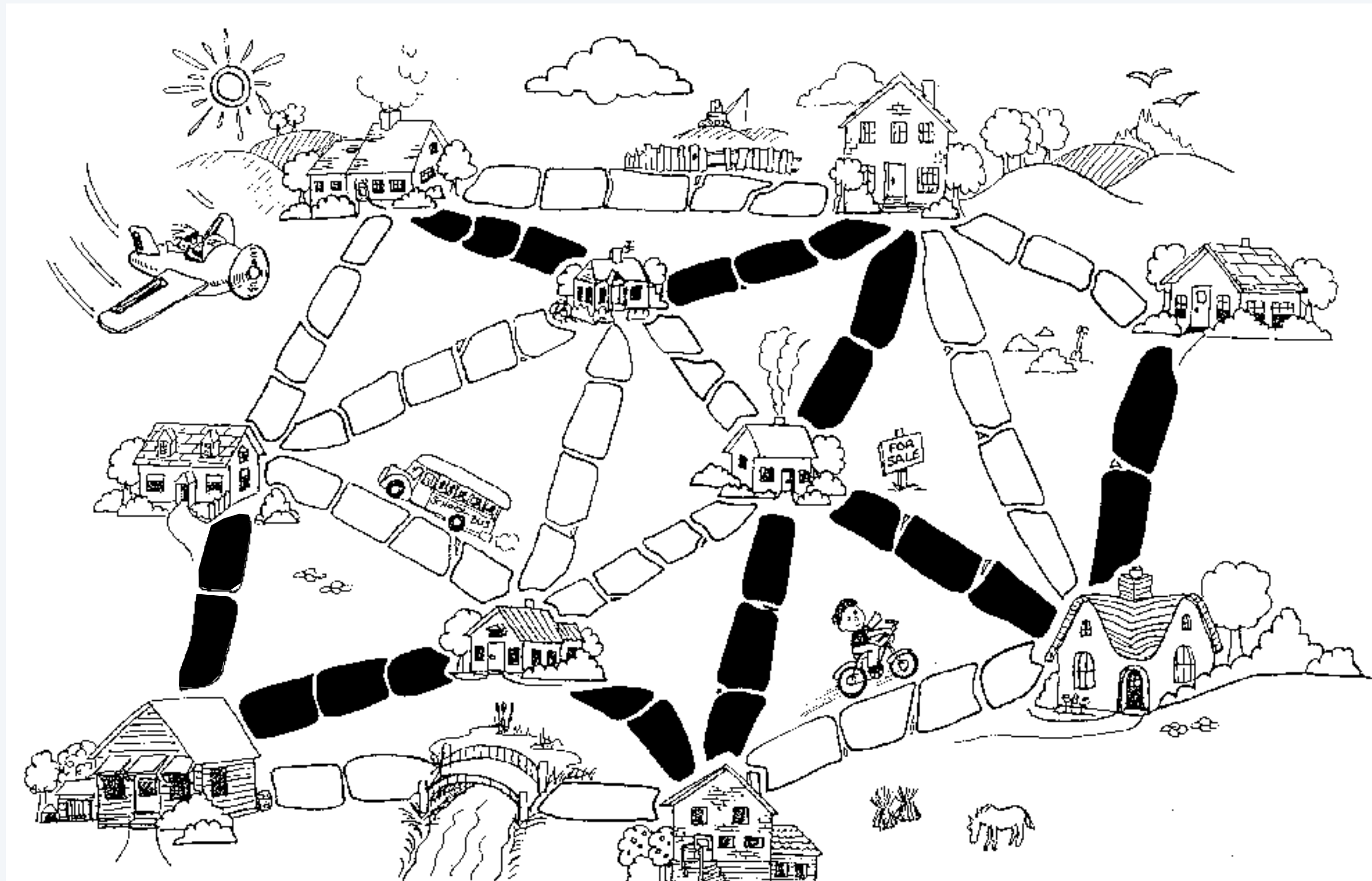


spanning tree T of graph G

Network design

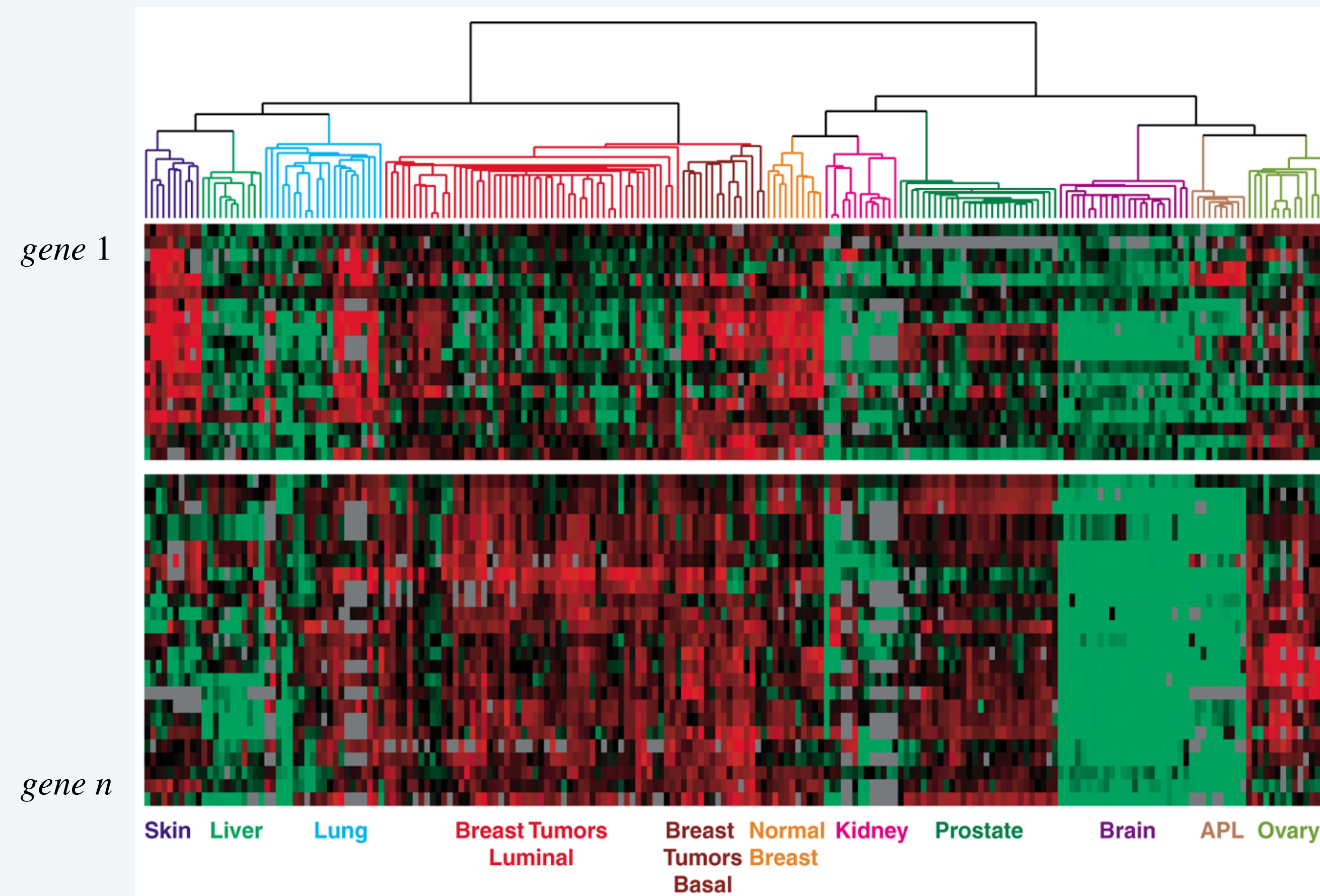
Network. Vertex = network component; edge = potential connection; edge weight = cost.

electrical, computer, telecommunication, transportation



Hierarchical clustering

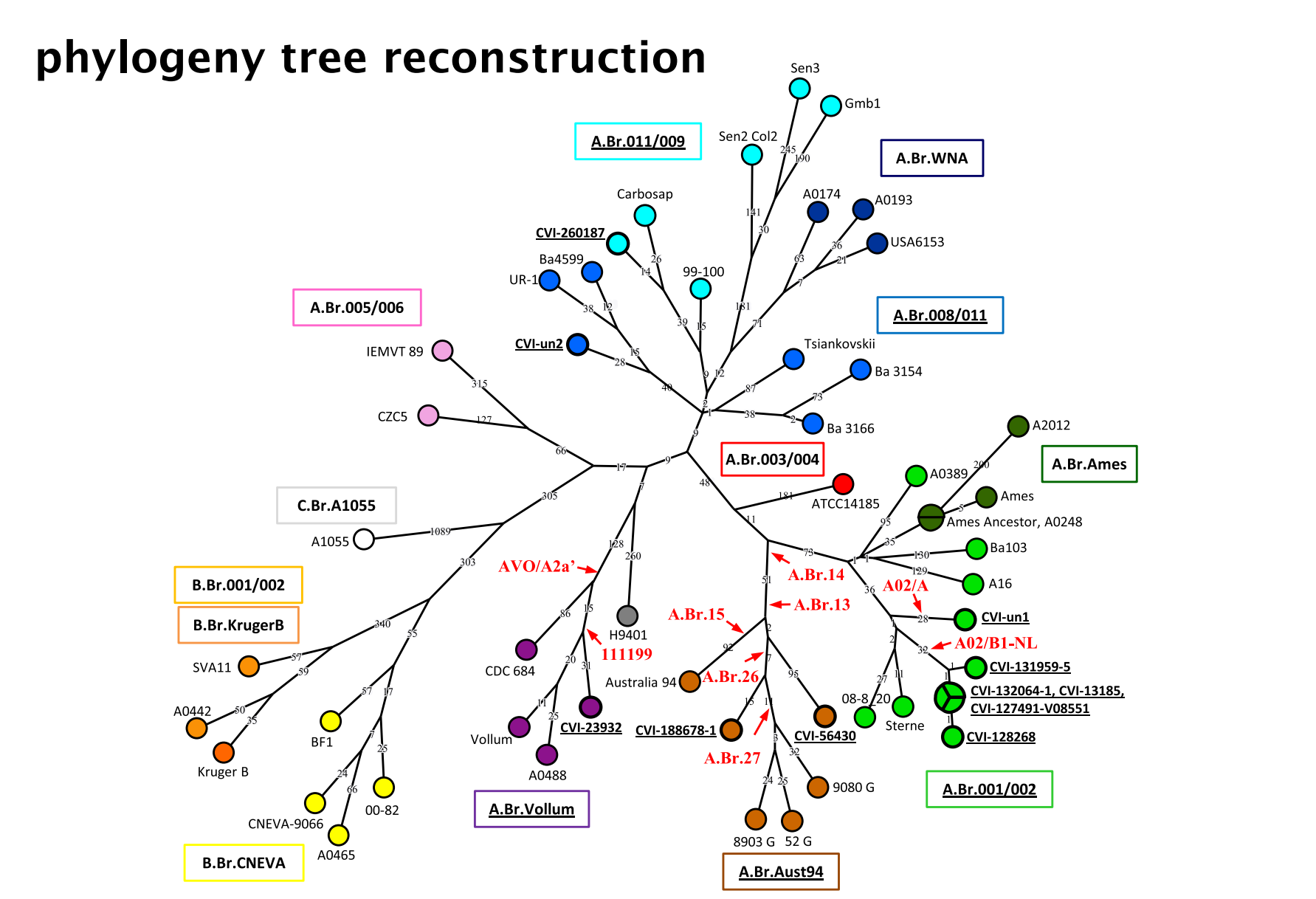
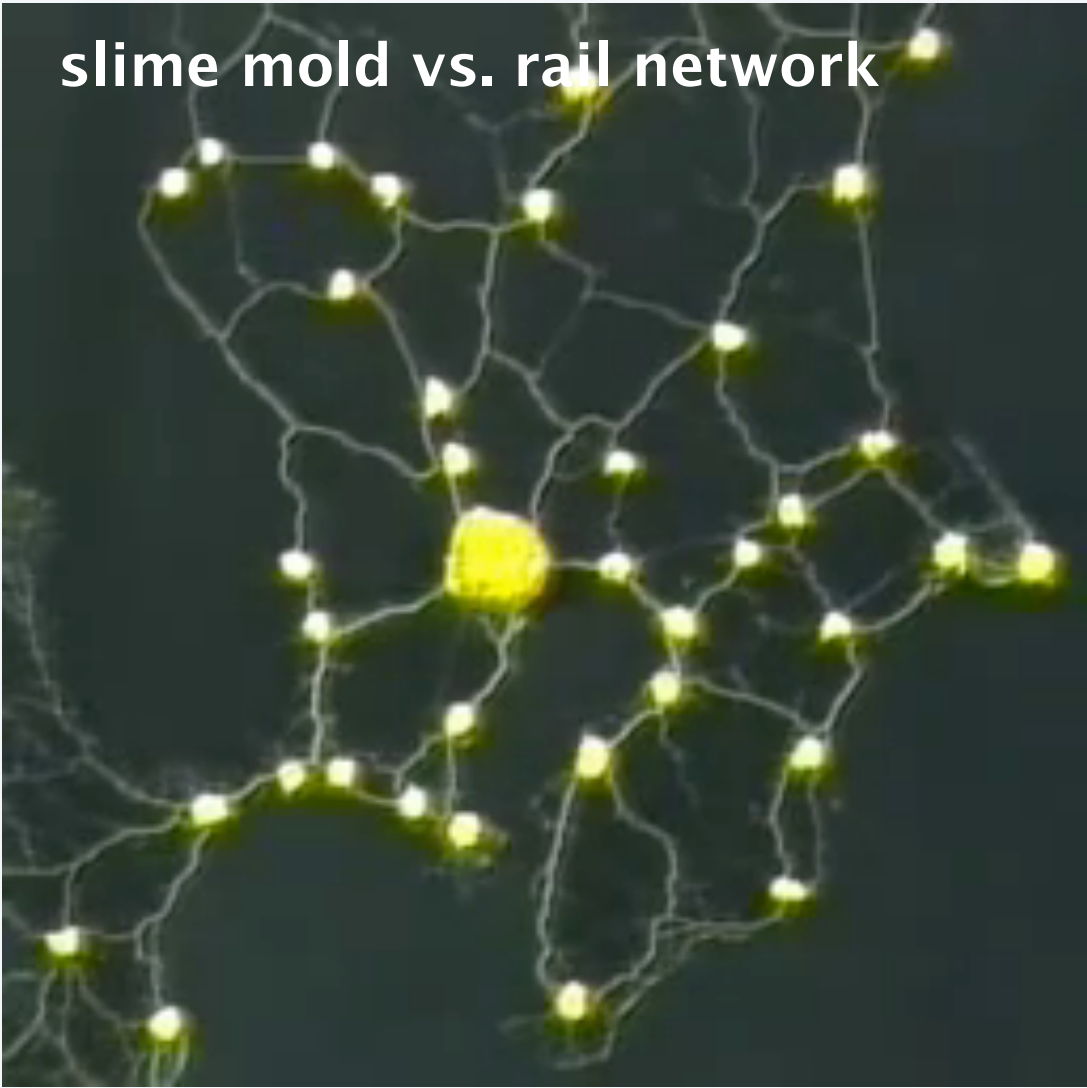
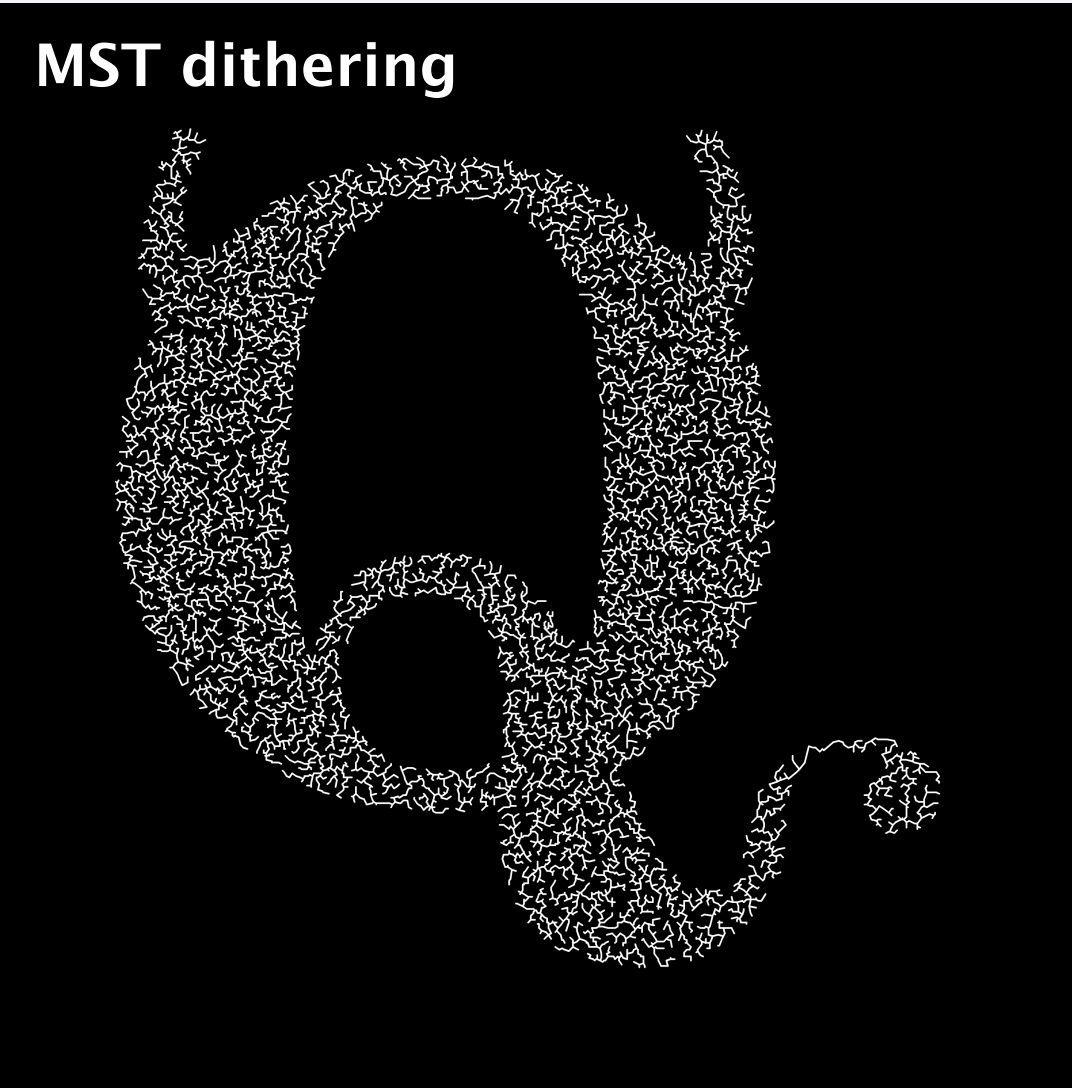
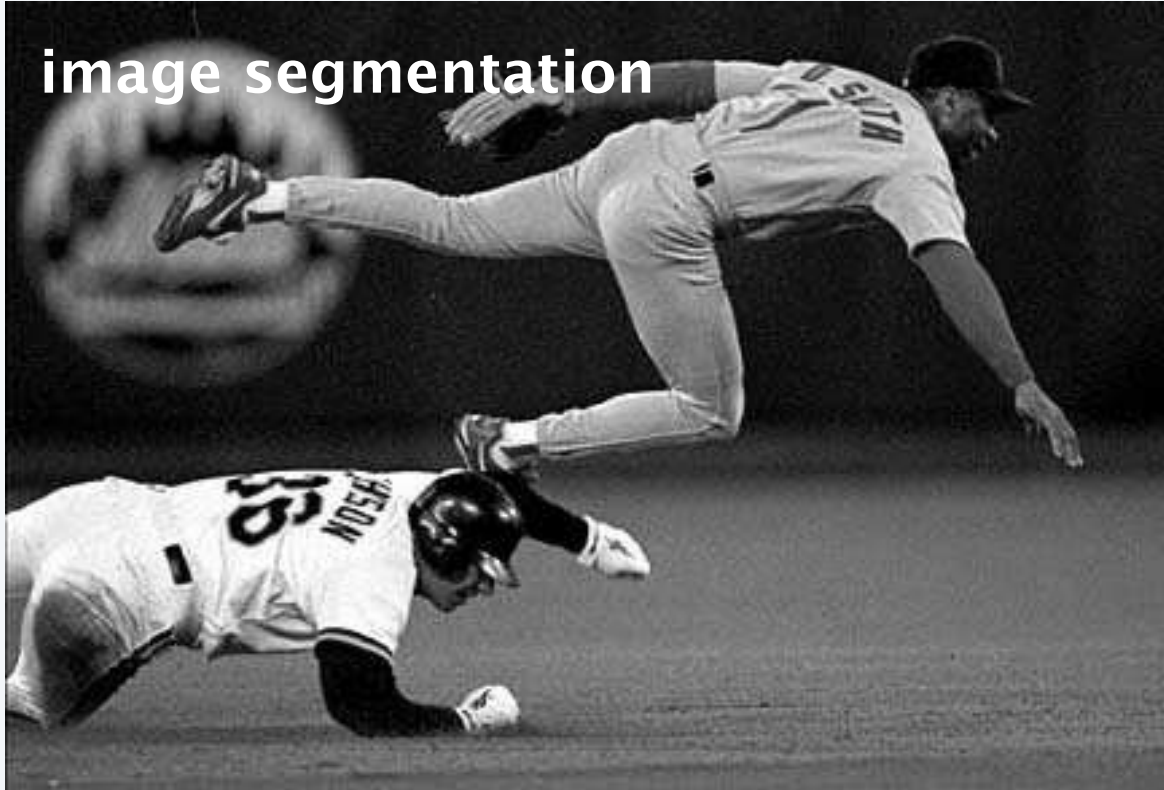
Microarray graph. Vertex = cancer tissue; edge = all pairs; edge weight = dissimilarity.

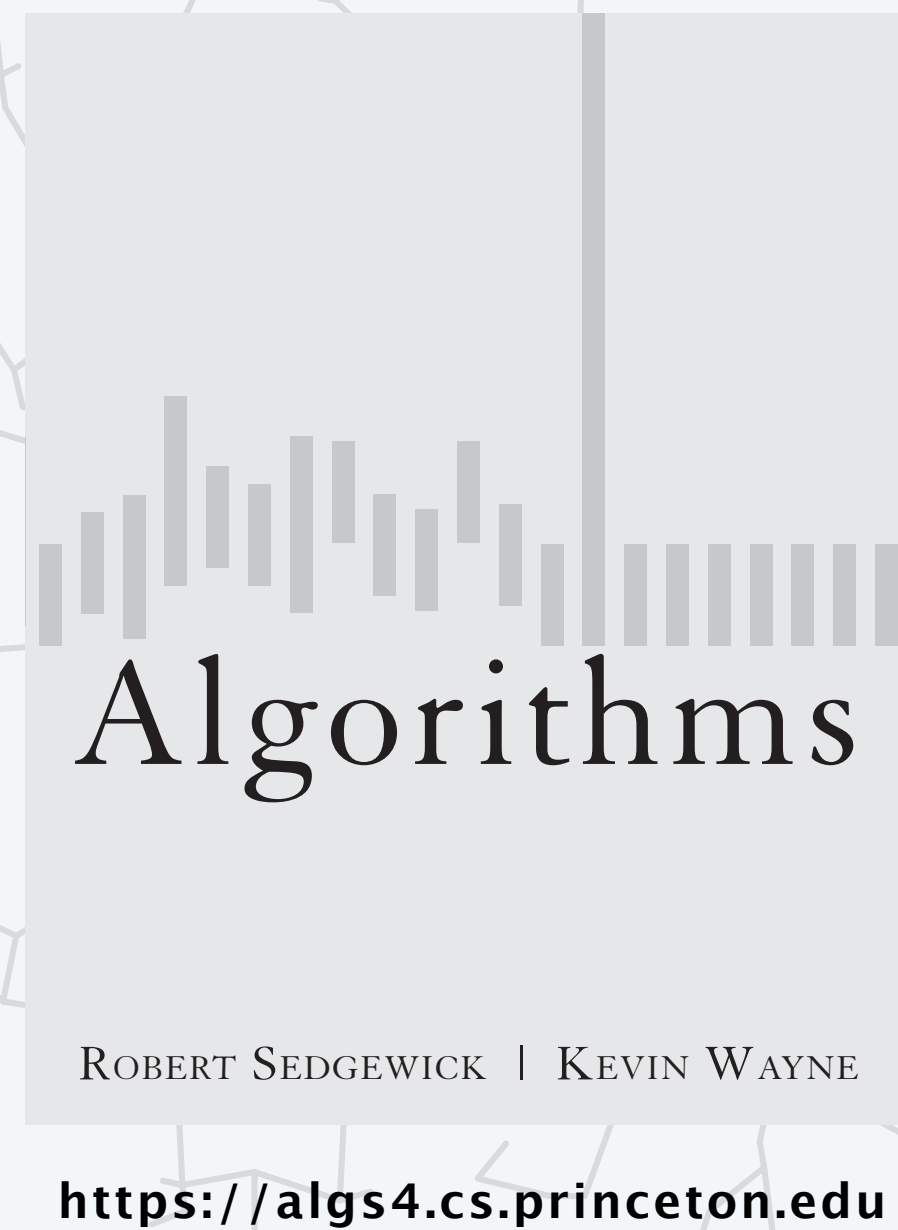


Reference: Botstein & Brown group

■ *gene expressed*
■ *gene not expressed*

More MST applications





4.3 MINIMUM SPANNING TREES

- *introduction*
- *cut property*
- *edge-weighted graph API*
- *Kruskal's algorithm*
- *Prim's algorithm*

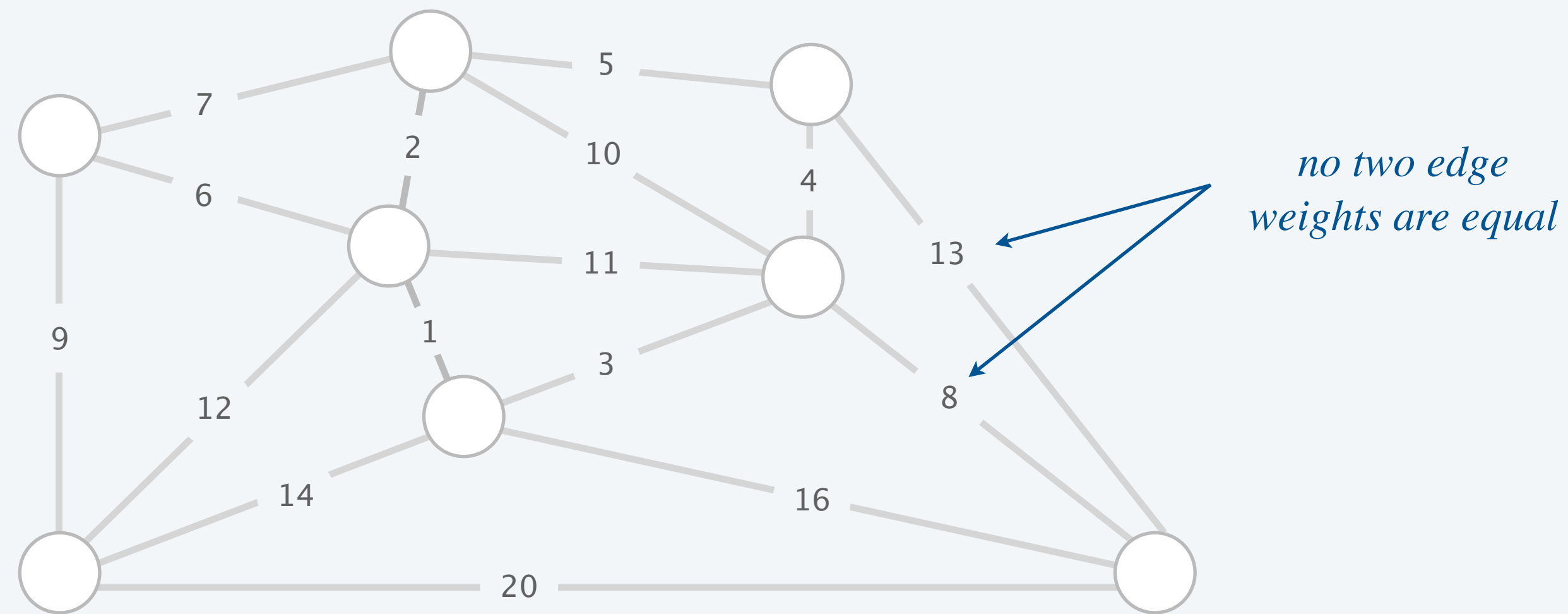
Simplifying assumptions

For simplicity, we assume:

- The graph is connected. $\Rightarrow$ MST exists.
- The edge weights are distinct. $\Rightarrow$ MST is unique. $\leftarrow$ *see Exercise 4.3.3 (solved on booksite)*

Note. Today's algorithms all work with duplicate edge weights.

but assumption simplifies the analysis



Cut property

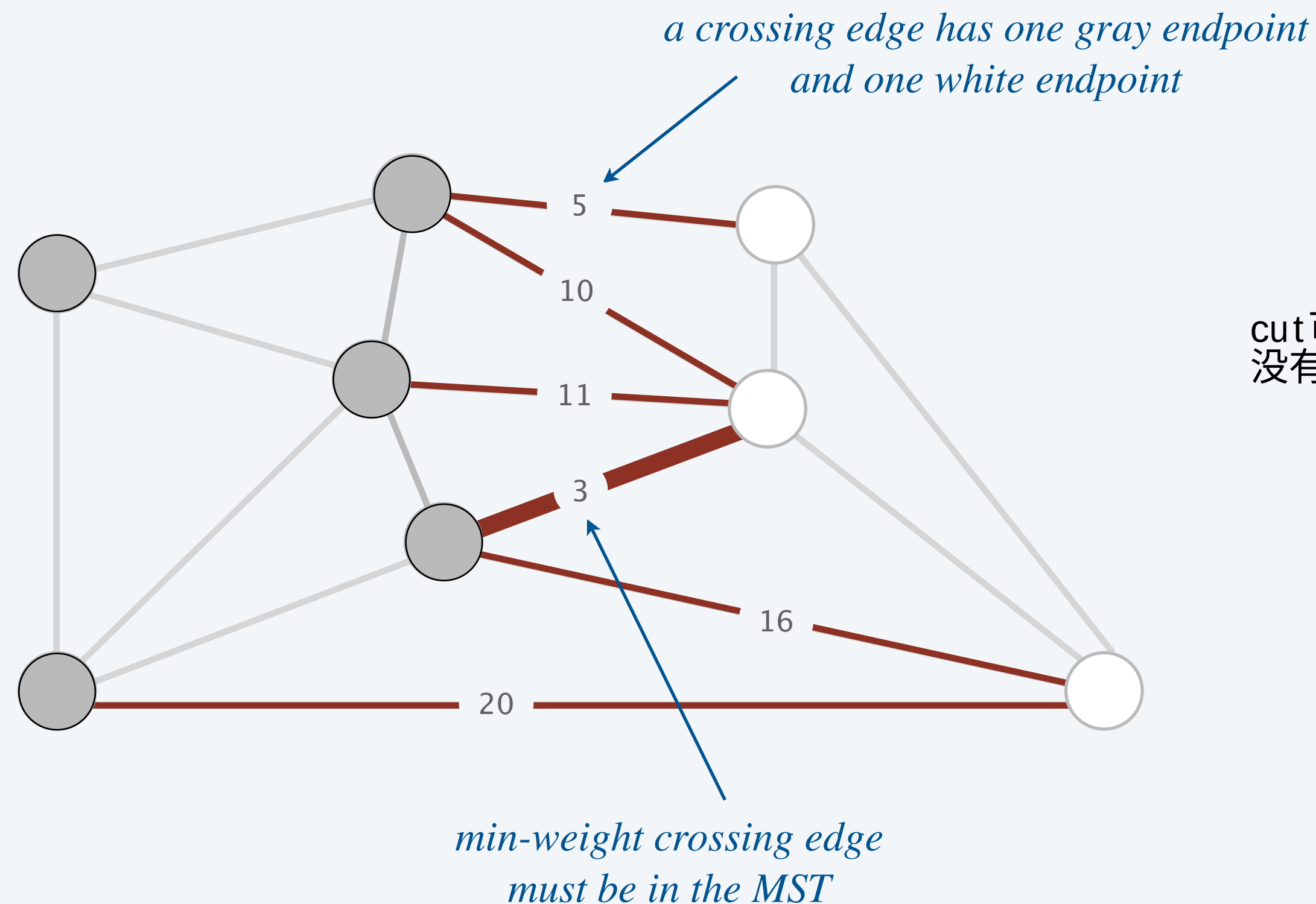
集合

非空

Def. A **cut** in a graph is a partition of its vertices into two nonempty sets.

Def. A **crossing edge** of a cut is an edge that has one endpoint in each set.

Cut property. For any cut, its min-weight crossing edge is in the MST.
min-spanning-tree
最小生成树



cut可以是任意的，只要是两个不相交集就行，和一个集合中顶点所在的位置没有关系

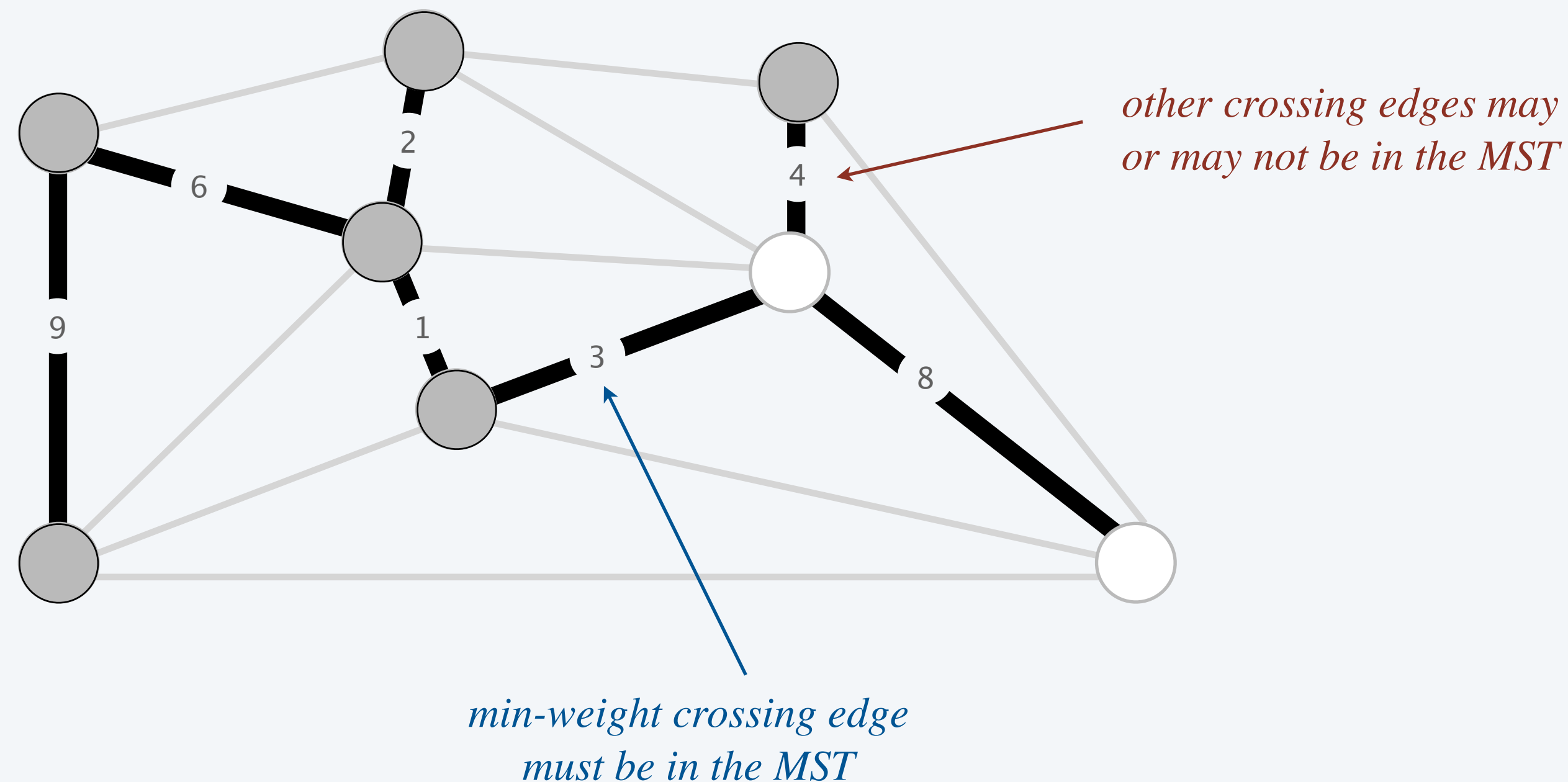
Cut property

Def. A **cut** in a graph is a partition of its vertices into two nonempty sets.

Def. A **crossing edge** of a cut is an edge that has one endpoint in each set.

Cut property. For any cut, its min-weight crossing edge is in the MST.

Note. A cut may have multiple edges in the MST.





Which is the min-weight edge crossing the cut { 2, 3, 5, 6 } ?

~~A. 0-7 (0.16)~~

~~B. 2-3 (0.17)~~

C. 0-2 (0.26)

D. 5-7 (0.28)

0-7 0.16

2-3 0.17

1-7 0.19

0-2 0.26

5-7 0.28

1-3 0.29

1-5 0.32

2-7 0.34

4-5 0.35

1-2 0.36

4-7 0.37

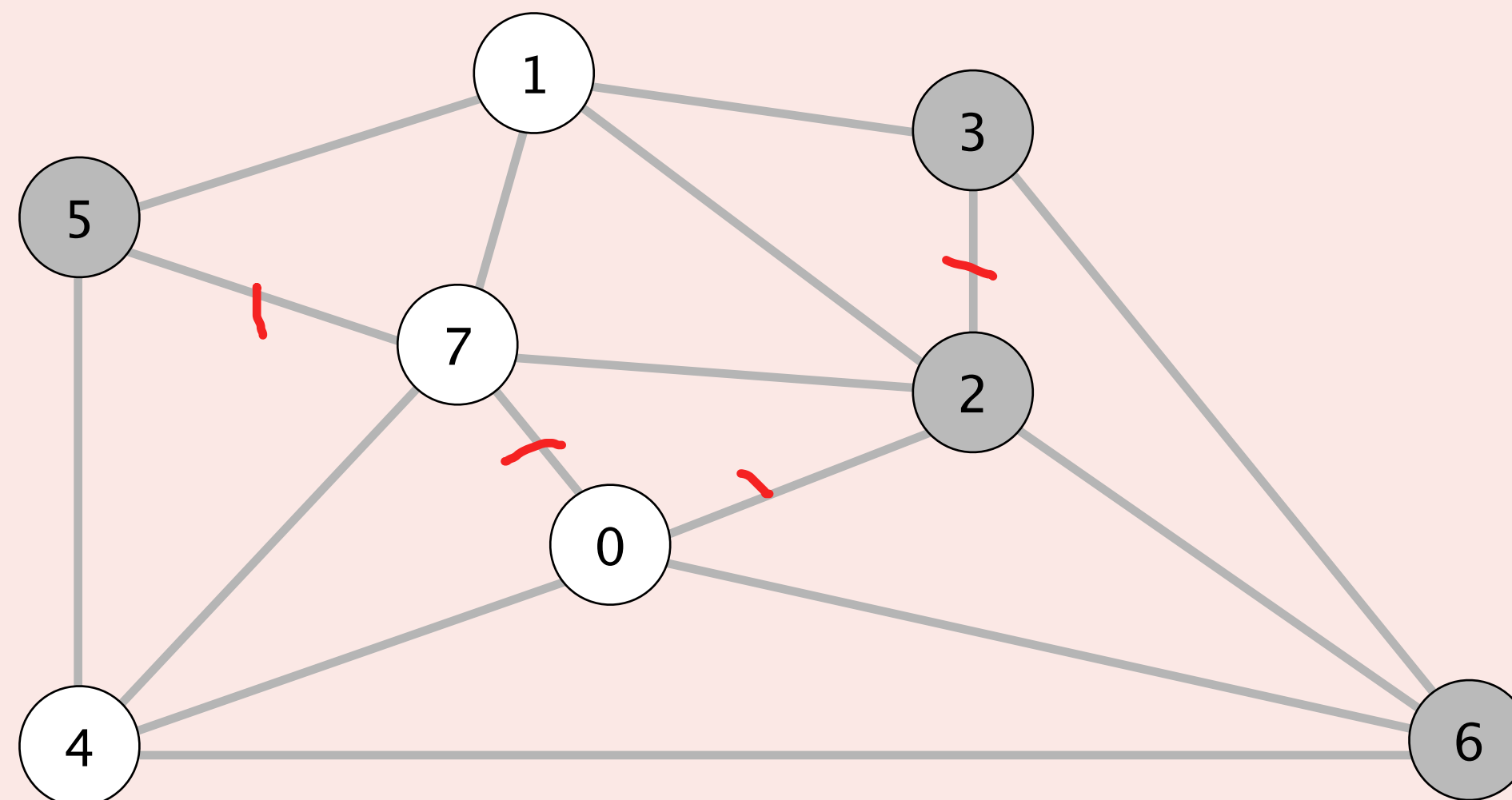
0-4 0.38

6-2 0.40

3-6 0.52

6-0 0.58

6-4 0.93



Cut property: correctness proof

Def. A **cut** in a graph is a partition of its vertices into two nonempty sets.

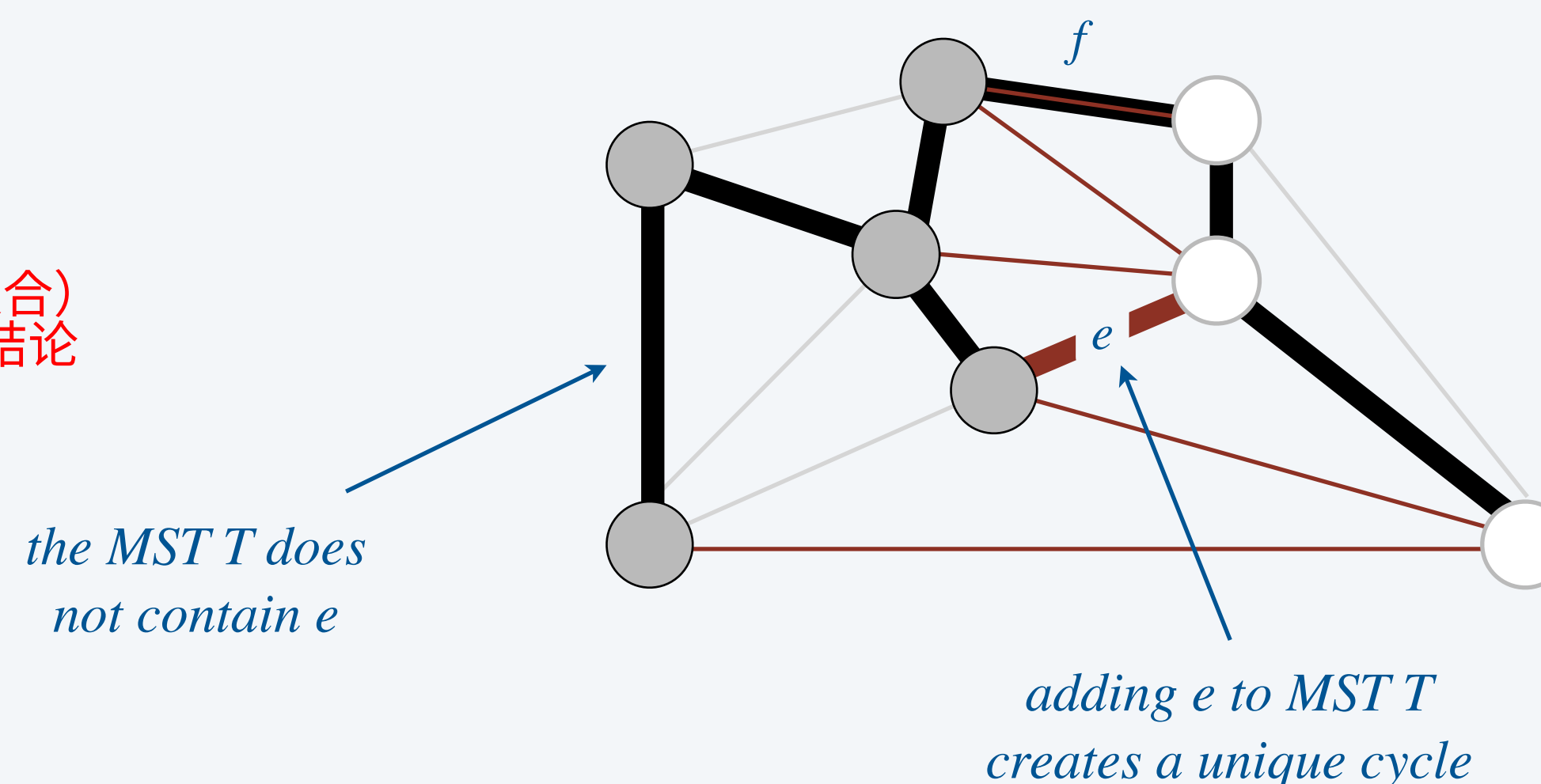
Def. A **crossing edge** of a cut is an edge that has one endpoint in each set.

 Cut property. For any cut, its min-weight crossing edge e is in the MST.

Pf. [by contradiction] Suppose e is not in the MST T .

- Adding e to T creates a unique cycle.
- Some other edge f in cycle must also be a crossing edge. 否则（无）是两个不连通的集合
- Removing f and adding e to T yields a different spanning tree T' . 隐形假设：所有的权重各不相同
- Since $weight(e) < weight(f)$, we have $weight(T') < weight(T)$.
- Contradiction. ■ 矛盾（反证法）

PS ;
如果cut划分比较混乱，比如 左1-中2-右1（1，2两个集合）
那么其实可以左-中和中-右分别看，分别证明上面这个结论



Framework for minimum spanning tree algorithms

Generic algorithm (to compute MST in G)

$T = \emptyset$.

Repeat until T is a spanning tree: $\longleftarrow V-1$ edges

- Find a cut in G .
- $e \leftarrow$ min-weight crossing edge.
- $T \leftarrow T \cup \{e\}$.

Efficient implementations.

- Which cut? $\longleftarrow 2^{V-2}$ distinct cuts
- How to compute min-weight crossing edge?

C_{V-1} 就是从 V 中取1个，注意 C_{V-1} 和 $CV1$ 是一样的，求和后要除以2

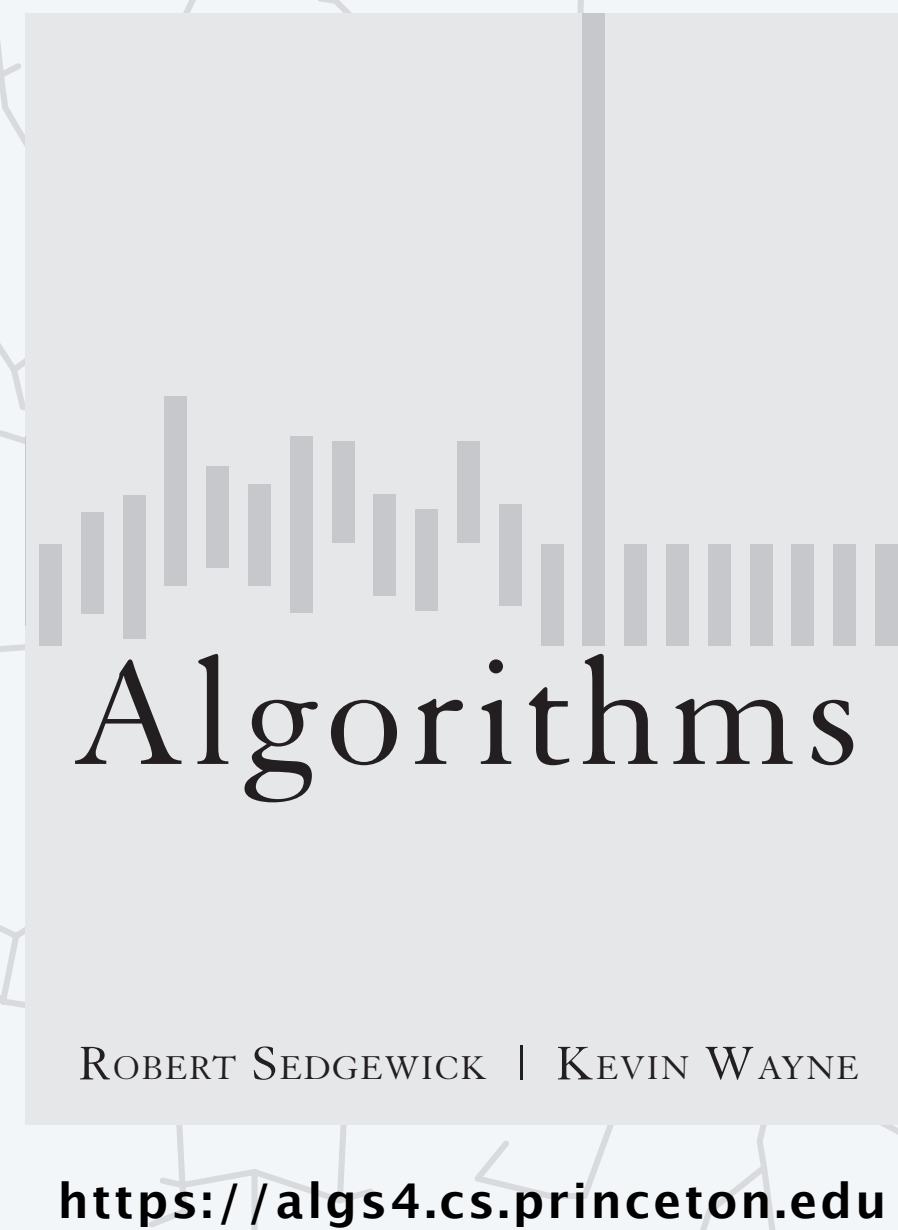
这是STcut的总数，确定两个顶点 st 一定是不相交的，其余顶点可以选择 s 或 t ，所以一共是 $V-2$ 个2相乘

另一种计算方式：每个节点都可以加入 A 或 B 集合，去掉全 A 全 B 两种，并考虑对称性，就是 $(2^{V-2}) / 2 = 2^{(V-1)-1}$

Ex 1. Kruskal's algorithm.

Ex 2. Prim's algorithm.

Ex 3. Borůvka's algorithm.



4.3 MINIMUM SPANNING TREES

- *introduction*
- *cut property*
- *edge-weighted graph API*
- *Kruskal's algorithm*
- *Prim's algorithm*

Weighted edge API

API. Edge abstraction for weighted edges.

```
public class Edge implements Comparable<Edge>
```

```
    Edge(int v, int w, double weight)    create a weighted edge v-w
```

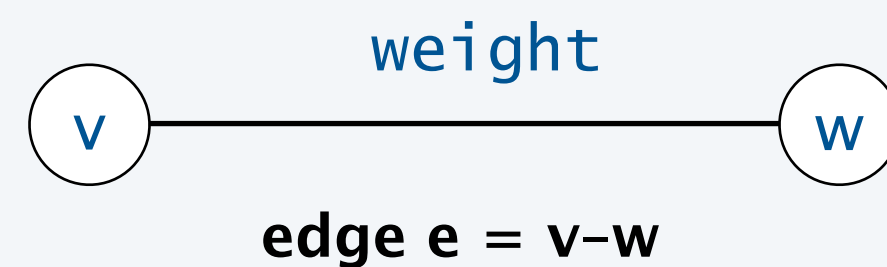
```
    int either()                        either endpoint
```

```
    int other(int v)                   the endpoint that's not v
```

```
    int compareTo(Edge that)          compare edges by weight
```

```
    ⋮
```

```
    ⋮
```



Idiom for processing an edge *e*. `int v = e.either(), w = e.other(v).`

Weighted edge: Java implementation

```
public class Edge implements Comparable<Edge> {  
    private final int v, w;  
    private final double weight;
```

```
    public Edge(int v, int w, double weight) {  
        this.v = v;  
        this.w = w;  
        this.weight = weight;  
    }
```

← *constructor*

```
    public int either() {  
        return v;  
    }
```

← *either endpoint*

```
    public int other(int vertex) {  
        if (vertex == v) return w;  
        else return v;  
    }
```

← *other endpoint*

```
    public int compareTo(Edge that) {  
        return Double.compare(this.weight, that.weight);  
    }
```

← *compare edges by weight*

```
}
```

Edge-weighted graph API

API. Same as [Graph](#) and [Digraph](#), except with explicit [Edge](#) objects.

```
public class EdgeWeightedGraph
```

EdgeWeightedGraph(int V)	<i>edge-weighted graph with V vertices (and no edges)</i>
--------------------------	---

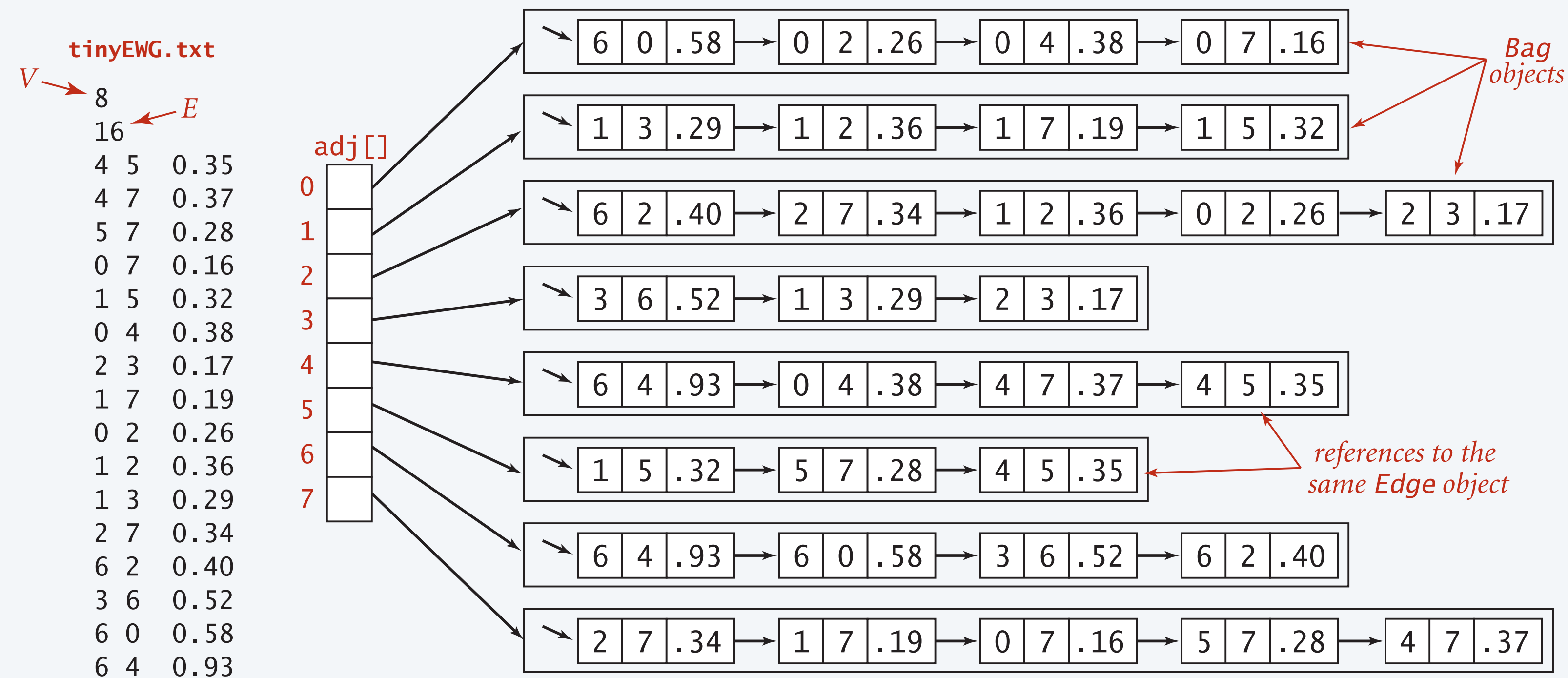
void	addEdge(Edge e)	<i>add weighted edge e to this graph</i>
------	-----------------	--

Iterable<Edge>	adj(int v)	<i>edges incident to v</i>
----------------	------------	----------------------------

:	:
---	---

Edge-weighted graph: adjacency-lists representation

Representation. Maintain vertex-indexed array of **Edge** lists.



Edge-weighted graph: adjacency-lists implementation

```
public class EdgeWeightedGraph {  
    private final int V;  
    private final Bag<Edge>[] adj;
```

← *same as Graph (but adjacency lists of Edge objects)*

```
    public EdgeWeightedGraph(int V) {  
        this.V = V;  
        adj = (Bag<Edge>[]) new Bag[V];  
        for (int v = 0; v < V; v++)  
            adj[v] = new Bag<>();  
    }
```

← *constructor*

```
    public void addEdge(Edge e) {  
        int v = e.either(), w = e.other(v);  
        adj[v].add(e);  
        adj[w].add(e);  
    }
```

← *add same Edge object to both adjacency lists*

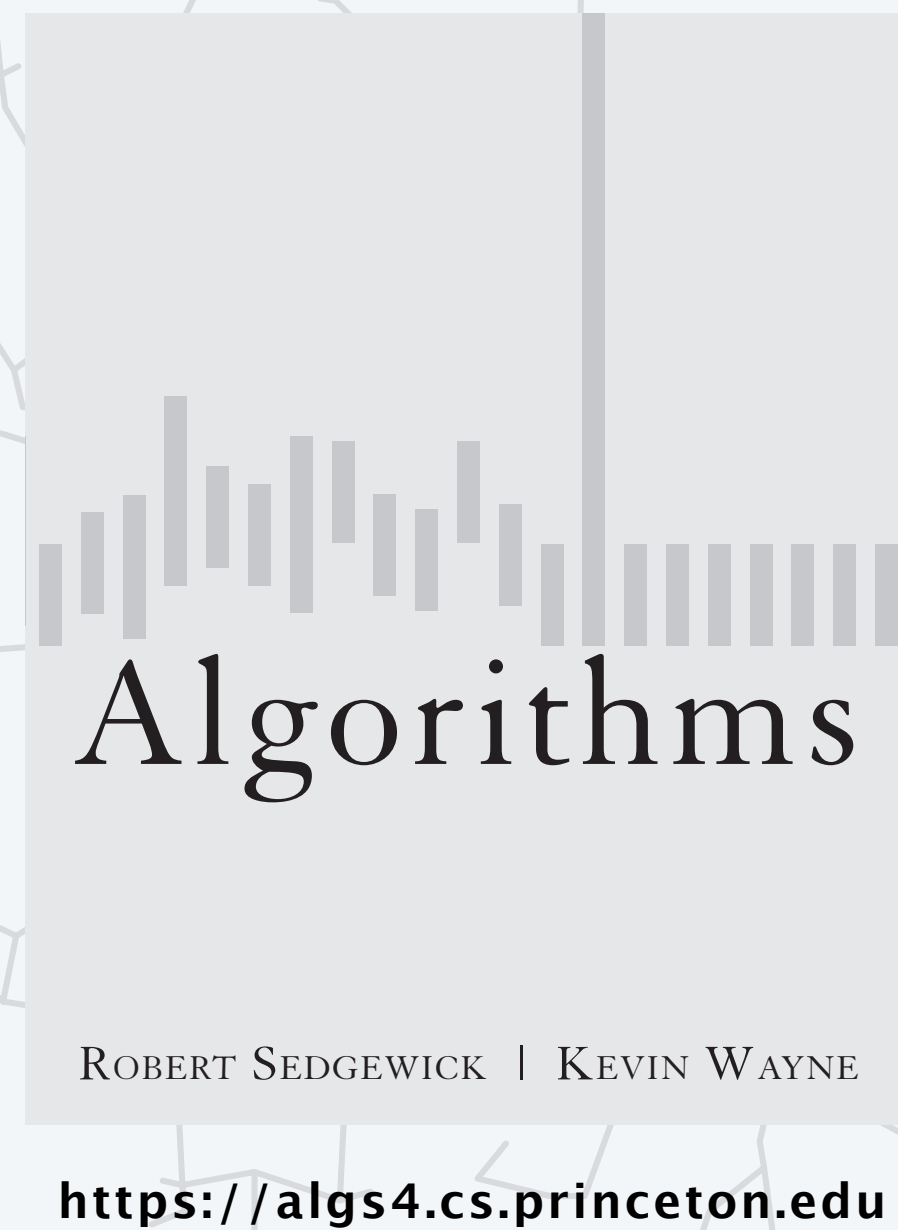
```
    public Iterable<Edge> adj(int v) {  
        return adj[v];  
    }
```

```
}
```

Minimum spanning tree API

- Q. How to represent the MST?
- A. Technically, an MST is an edge-weighted graph.
For convenience, we represent it as a set of edges.

public class MST		
	MST(EdgeWeightedGraph G)	<i>constructor</i>
Iterable<Edge>	edges()	<i>edges in MST</i>
double	weight()	<i>weight of MST</i>
	⋮	⋮



4.3 MINIMUM SPANNING TREES

- *introduction*
- *cut property*
- *edge-weighted graph API*
- *Kruskal's algorithm*
- *Prim's algorithm*

Kruskal's algorithm demo

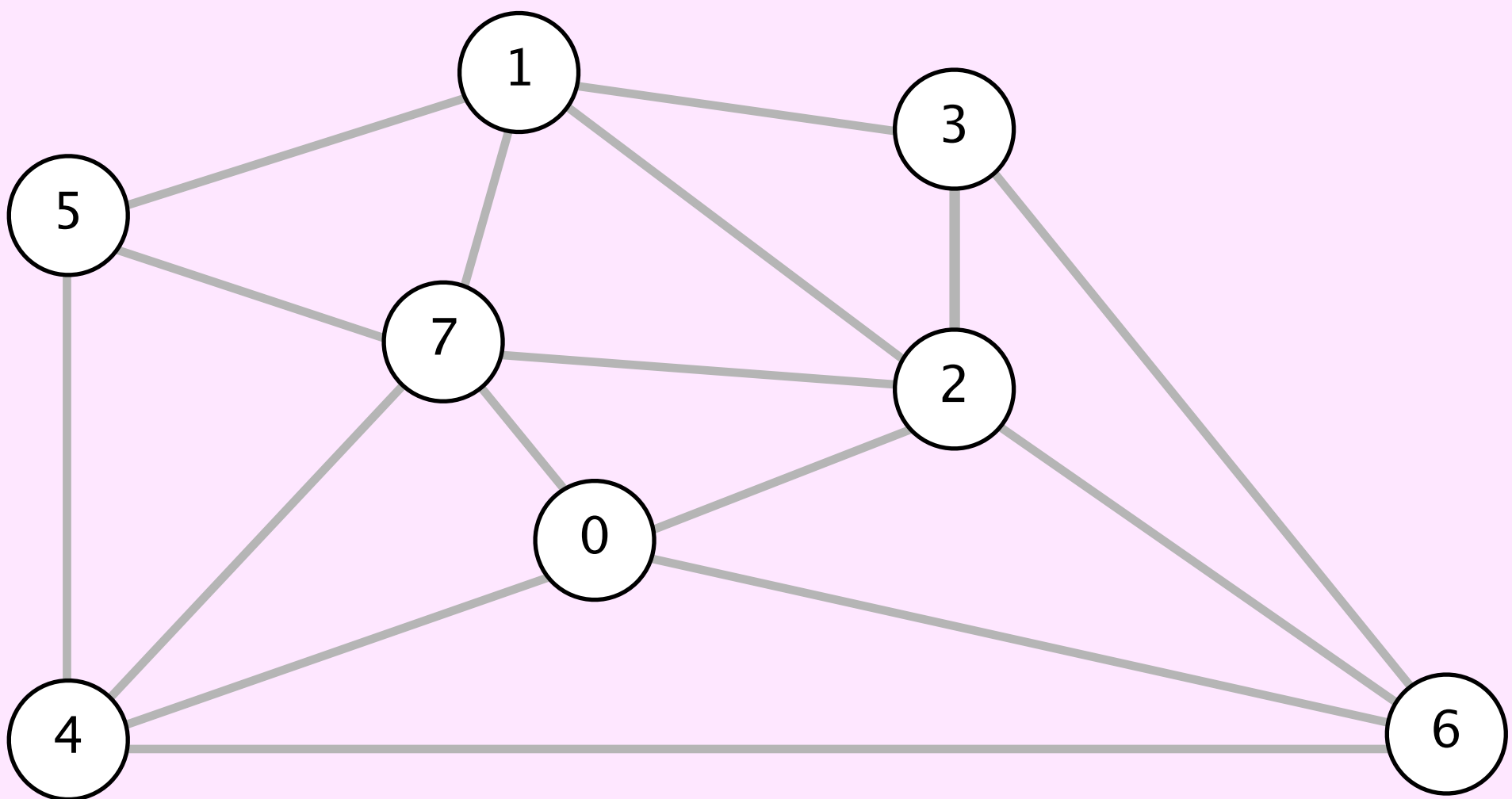
这是基于一组已经拍好序的边才能做的



递增的

Consider edges in ascending order of weight.

- Add next edge to T unless doing so would create a cycle.

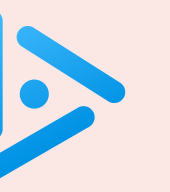


an edge-weighted graph

*graph edges
sorted by weight*

↓

0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93



In which order does Kruskal's algorithm select edges in MST?

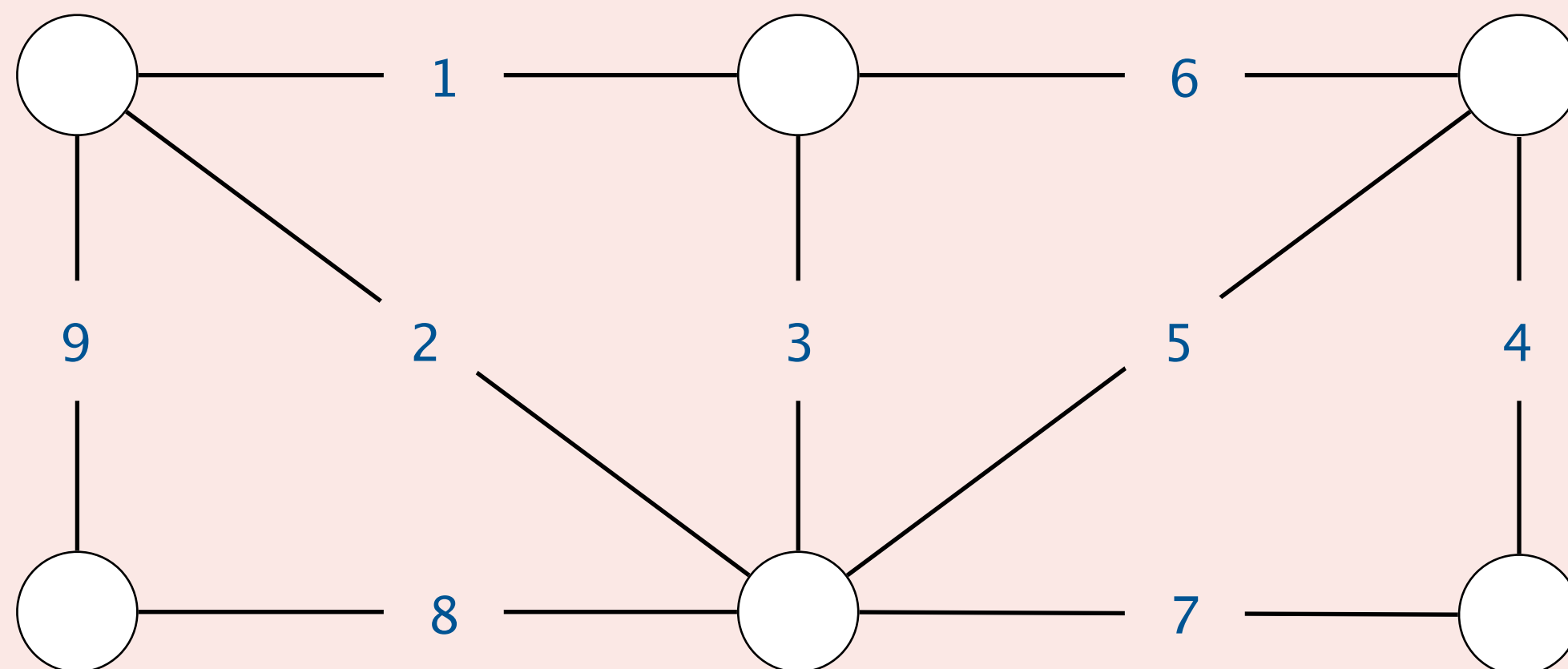
A. 1, 2, 4, 5, 6

一直取最小，直到产生环

☒ B. 1, 2, 4, 5, 8

C. 1, 2, 5, 4, 8

D. 8, 2, 1, 5, 4



Kruskal's algorithm: correctness proof

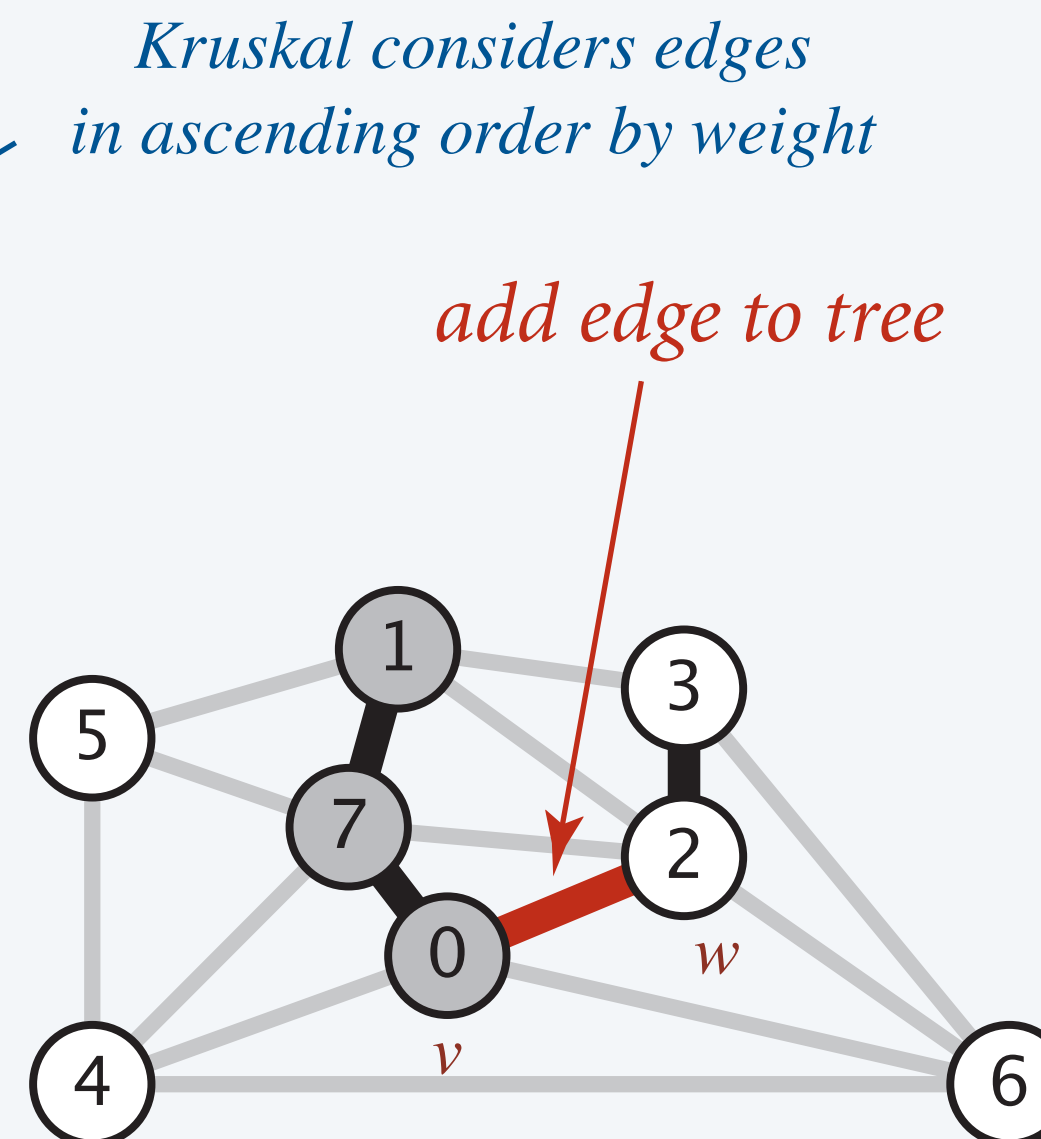
Proposition. [Kruskal 1956] Kruskal's algorithm computes the MST.

Pf. Kruskal's algorithm adds edge e to T if and only if e is in the MST.

[Case 1 $\Rightarrow$] Kruskal's algorithm adds edge $e = v-w$ to T .

- Vertices v and w are in different connected components of T .
- Cut = set of vertices connected to v in T .
- By construction of cut, e is a crossing edge and no crossing edge
 - is currently in T
 - was considered by Kruskal before e
- Thus, e is a min weight crossing edge.
- Cut property $\Rightarrow e$ is in the MST.

新加入的边，其两个顶点一定在不同的连通分量里面
否则就成环了



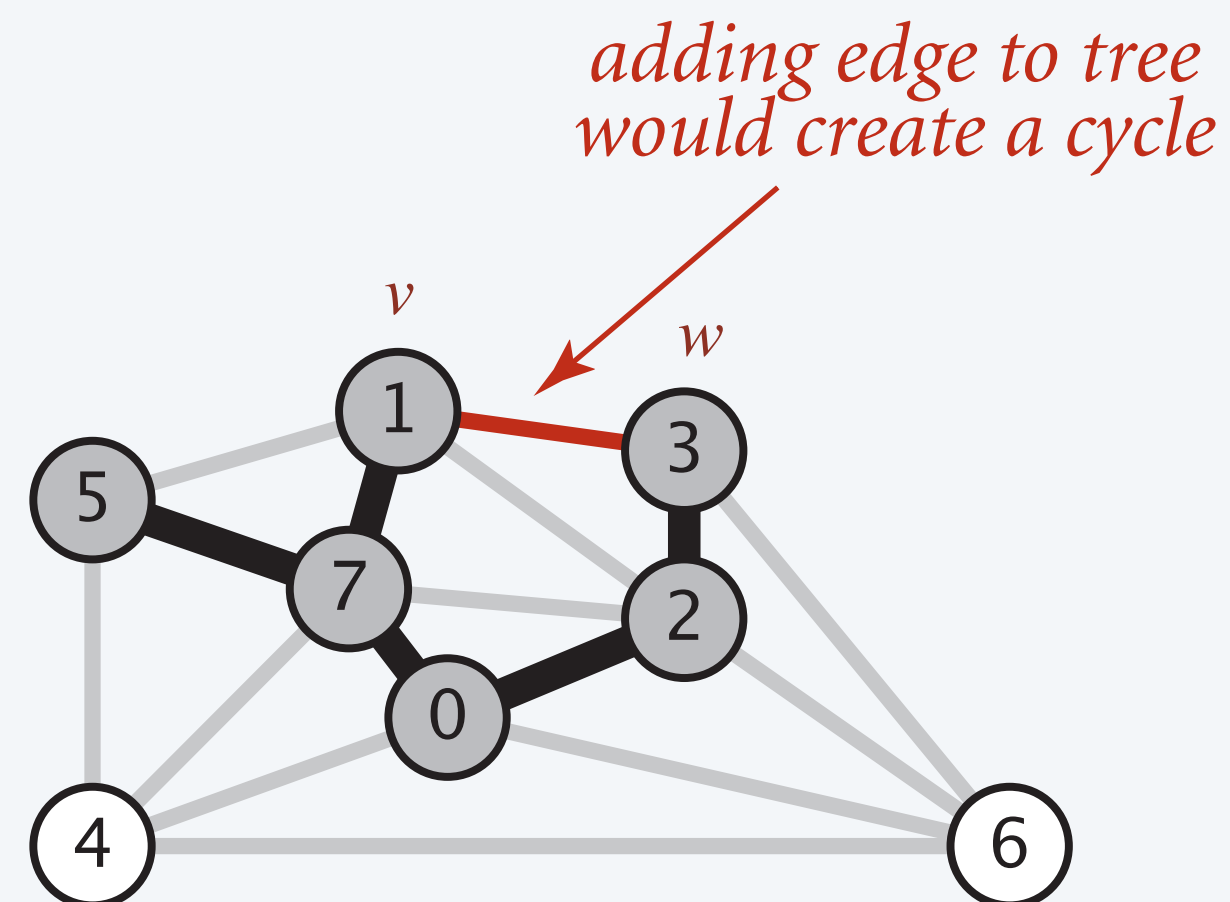
Kruskal's algorithm: correctness proof

Proposition. [Kruskal 1956] Kruskal's algorithm computes the MST.

Pf. Kruskal's algorithm adds edge e to T if and only if e is in the MST.

[Case 2 $\Leftarrow$] Kruskal's algorithm discards edge $e = v-w$.

- From Case 1, all edges currently in T are in the MST.
- The MST can't contain a cycle, so it can't also contain e . ■

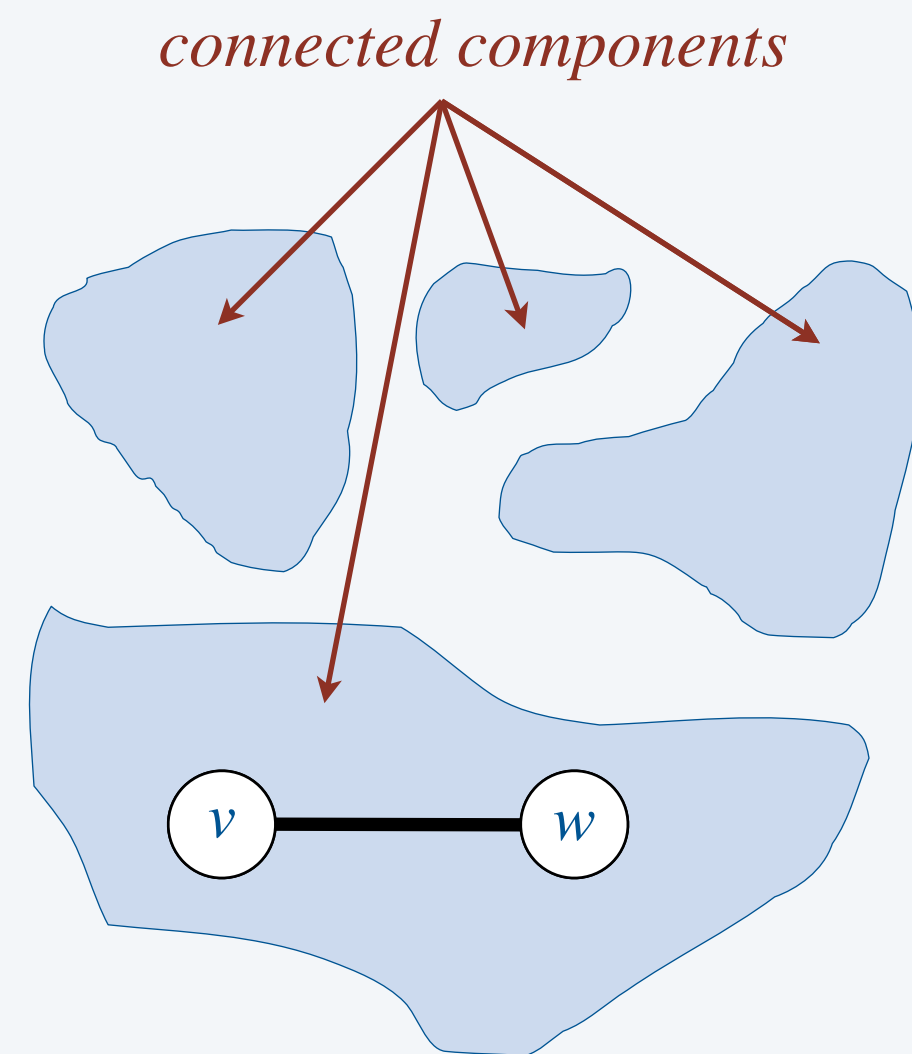


Kruskal's algorithm: implementation challenge

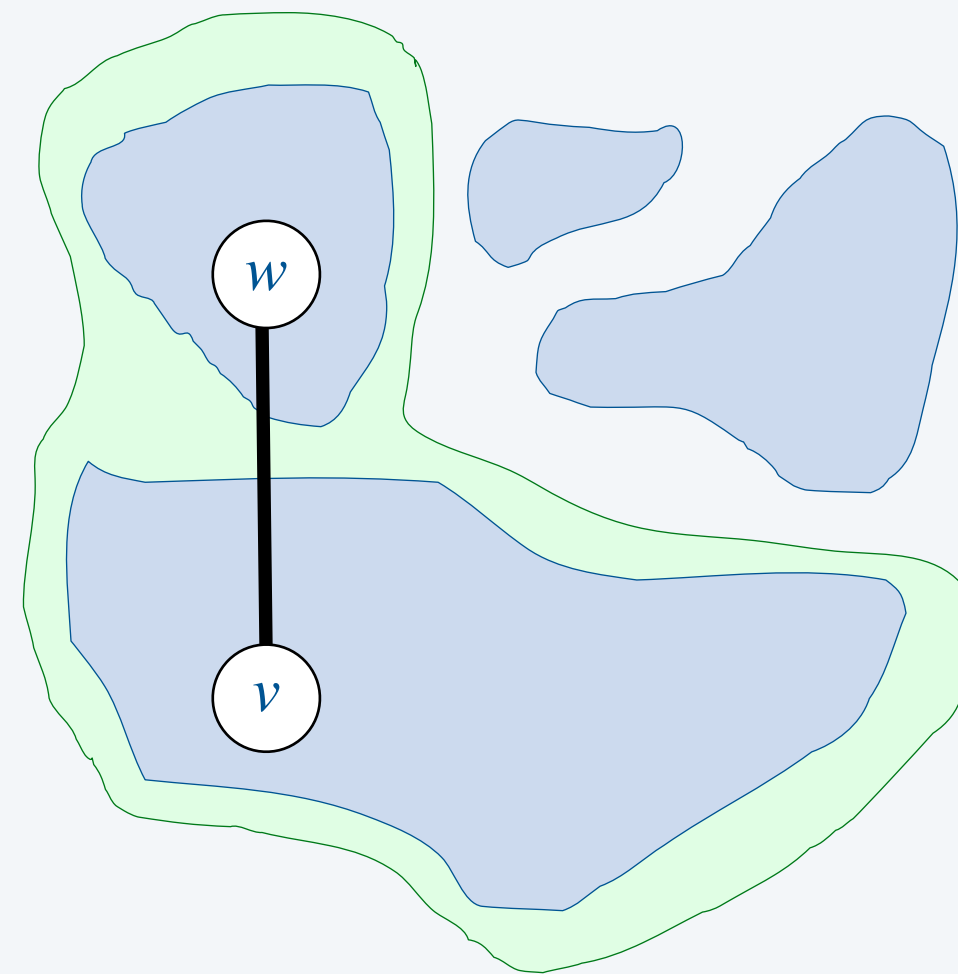
Challenge. Would adding edge $v-w$ to T create a cycle? If not, add it.

Efficient solution. Use the **union-find** data structure.

- Maintain a set for each **connected component** in T , initially each vertex in its own set.
- If v and w are in same set, then adding $v-w$ to T would create a cycle. [Case 2]
- Otherwise, add $v-w$ to T and merge sets containing v and w . [Case 1]



Case 2: adding $v-w$ creates a cycle



Case 1: add $v-w$ to T and merge sets containing v and w

Kruskal's algorithm: Java implementation

```
public class KruskalMST {
    private Queue<Edge> mst = new Queue<>();

    public KruskalMST(EdgeWeightedGraph G) {
        Edge[] edges = G.edges();
        Arrays.sort(edges);
        UF uf = new UF(G.V());

        for (int i = 0; i < G.E(); i++) {
            Edge e = edges[i];
            int v = e.either(), w = e.other(v);
            if (uf.find(v) != uf.find(w)) {
                mst.enqueue(e);
                uf.union(v, w);
            }
        }
    }

    public Iterable<Edge> edges() {
        return mst;
    }
}
```

← *edges in the MST*

← *sort edges by weight*

← *maintain connected components*

← *optimization: stop as soon as $V-1$ edges in T*

← *greedily add edges to MST*

← *edge $v-w$ does not create cycle*

← *add edge e to MST*

← *merge connected components*

Kruskal's algorithm: running time

Proposition. In the worst case, Kruskal's algorithm computes the MST in an edge-weighted graph in $\Theta(E \log E)$ time and $\Theta(E)$ extra space.

Pf.

包含了对边进行排序的集合，和Unionfind的空间
二者加起来是E+V的开销，因为V小于E，所以还是E

所谓extra SPACE，就是除了基本存储原始数据（这里是图G）之外所用到的空间

- Bottlenecks are sorting and union-find operations.

次数 每次的开销

operation	frequency	time per op
SORT	1	$E \log E$
UNION	$V - 1$	$\log V^\dagger$
FIND	$2 E$	$\log V^\dagger$

每条边的2个节点
分别find

† using weighted quick union

查找的结构类似一个二叉树的树高

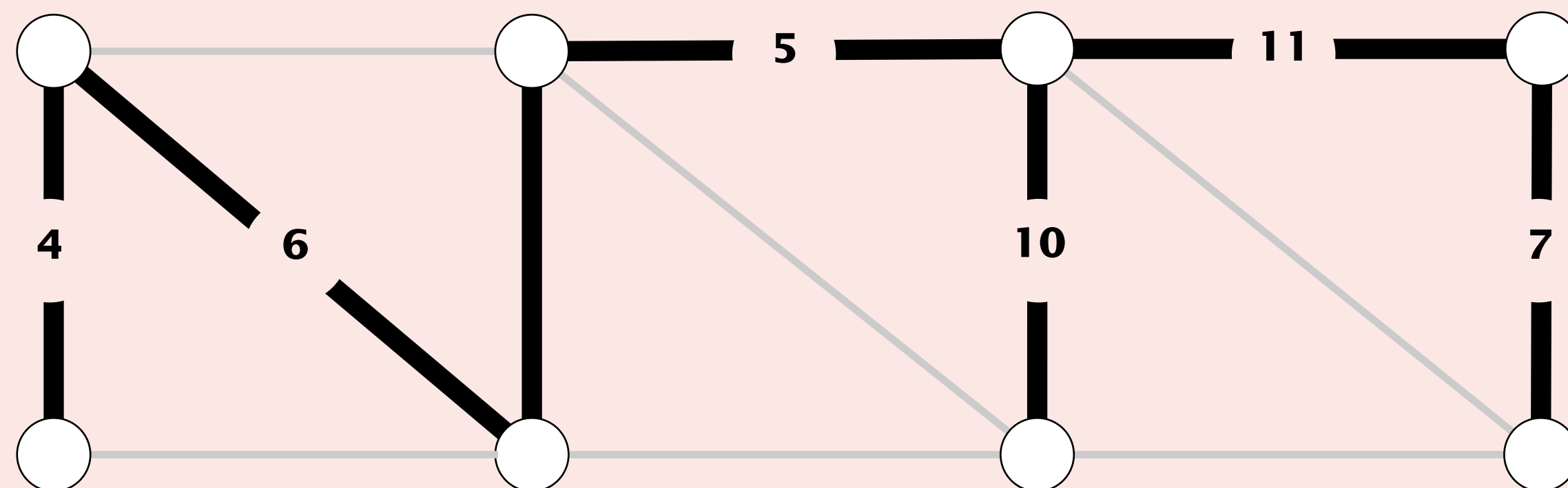
- Total. $\Theta(V \log V) + \Theta(E \log V) + \Theta(E \log E)$.

dominated by $\Theta(E \log E)$
since graph is connected

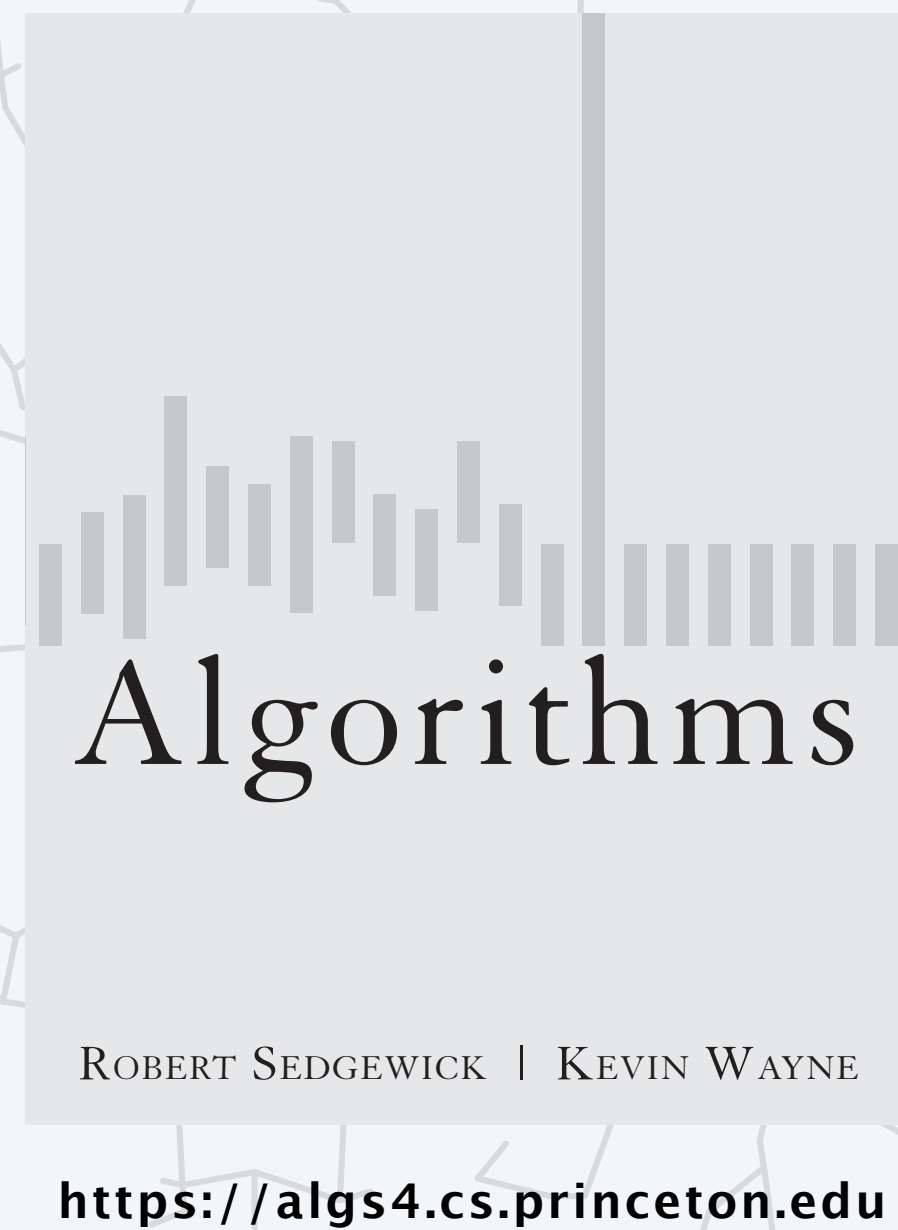


Given a graph with positive edge weights, how to find a spanning tree that minimizes the sum of the **squares** of the edge weights?

- A. Run Kruskal's algorithm using the **original** edge weights.
- B. Run Kruskal's algorithm using the **squares** of the edge weights. square : 平方
square root : 平方根
- C. Run Kruskal's algorithm using the **square roots** of the edge weights.
- D. All of the above.

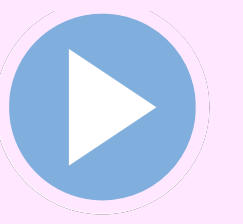


$$\text{sum of squares} = 4^2 + 6^2 + 5^2 + 10^2 + 11^2 + 7^2 = 347$$



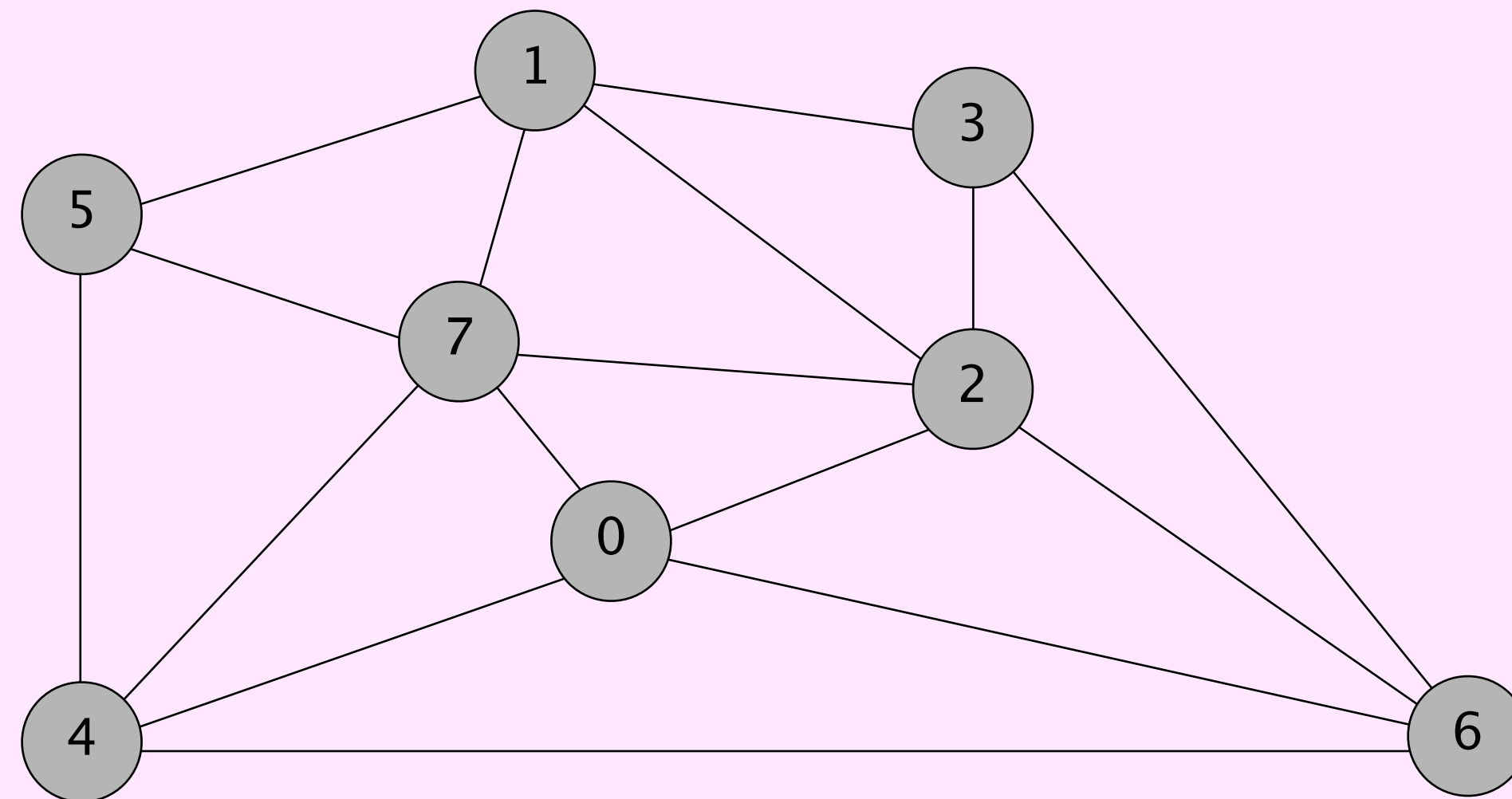
4.3 MINIMUM SPANNING TREES

- *introduction*
- *cut property*
- *edge-weighted graph API*
- *Kruskal's algorithm*
- *Prim's algorithm*



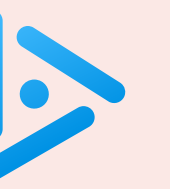
- Start with vertex 0 and grow tree T .
- Repeat until $V - 1$ edges:
 - add to T the min-weight edge with exactly one endpoint in T

核心是要选定一个起始节点



an edge-weighted graph

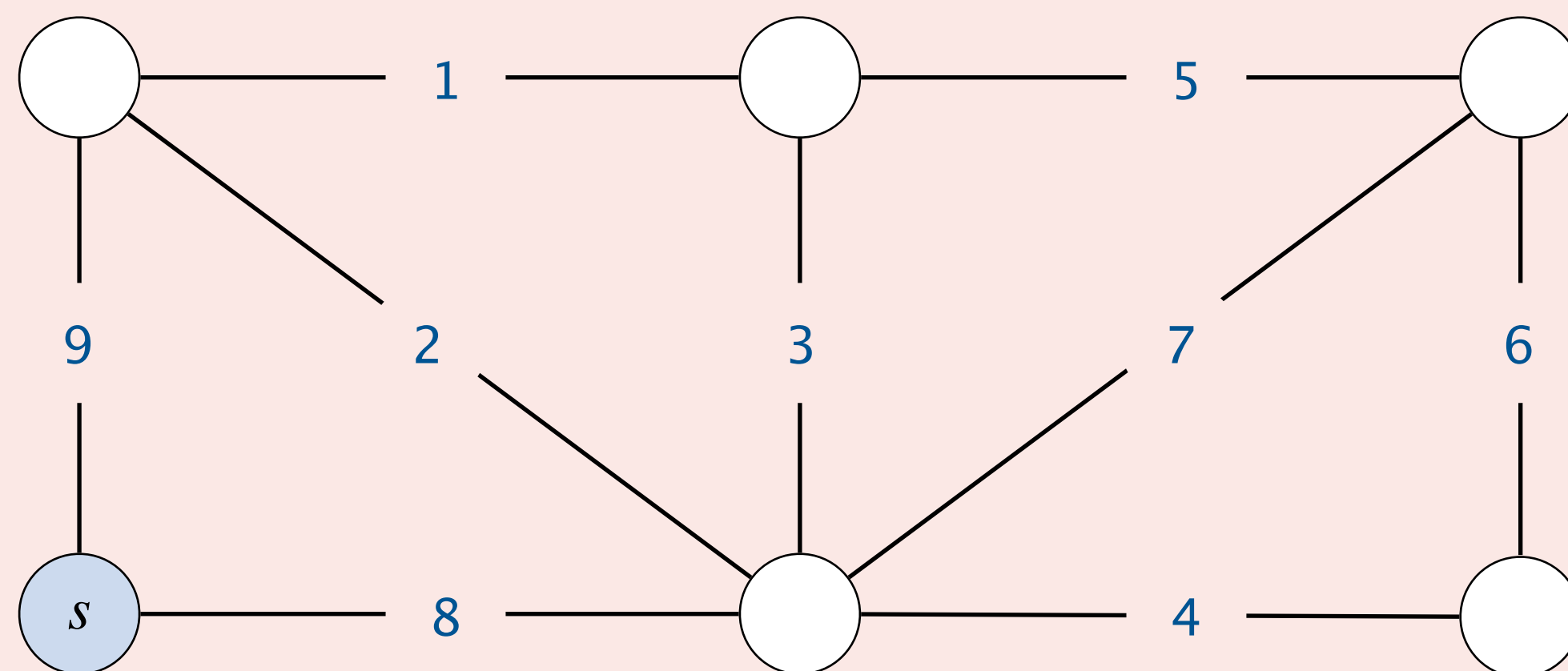
0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93



In which order does Prim's algorithm select edges in the MST?

Assume it starts from vertex s .

- A.** 8, 2, 1, 4, 5
- B.** 8, 2, 1, 5, 4
- C.** 8, 2, 1, 5, 6
- D.** 8, 2, 3, 4, 5



Prim's algorithm: proof of correctness

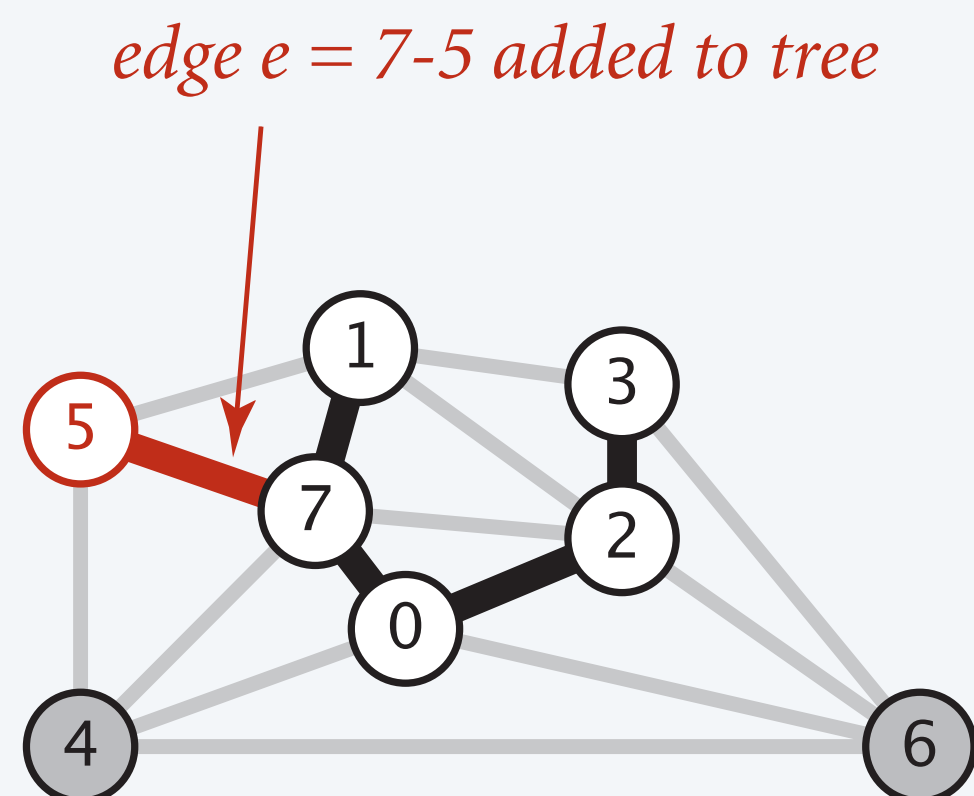
Proposition. [Jarník 1930, Dijkstra 1957, Prim 1959]

Prim's algorithm computes the MST.

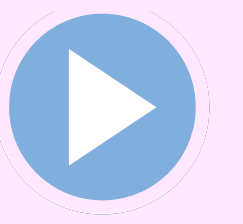
Pf. Let e = min-weight edge with exactly one endpoint in T .

- Cut = set of vertices in T .
- Cut property $\Rightarrow$ edge e is in the MST. ■

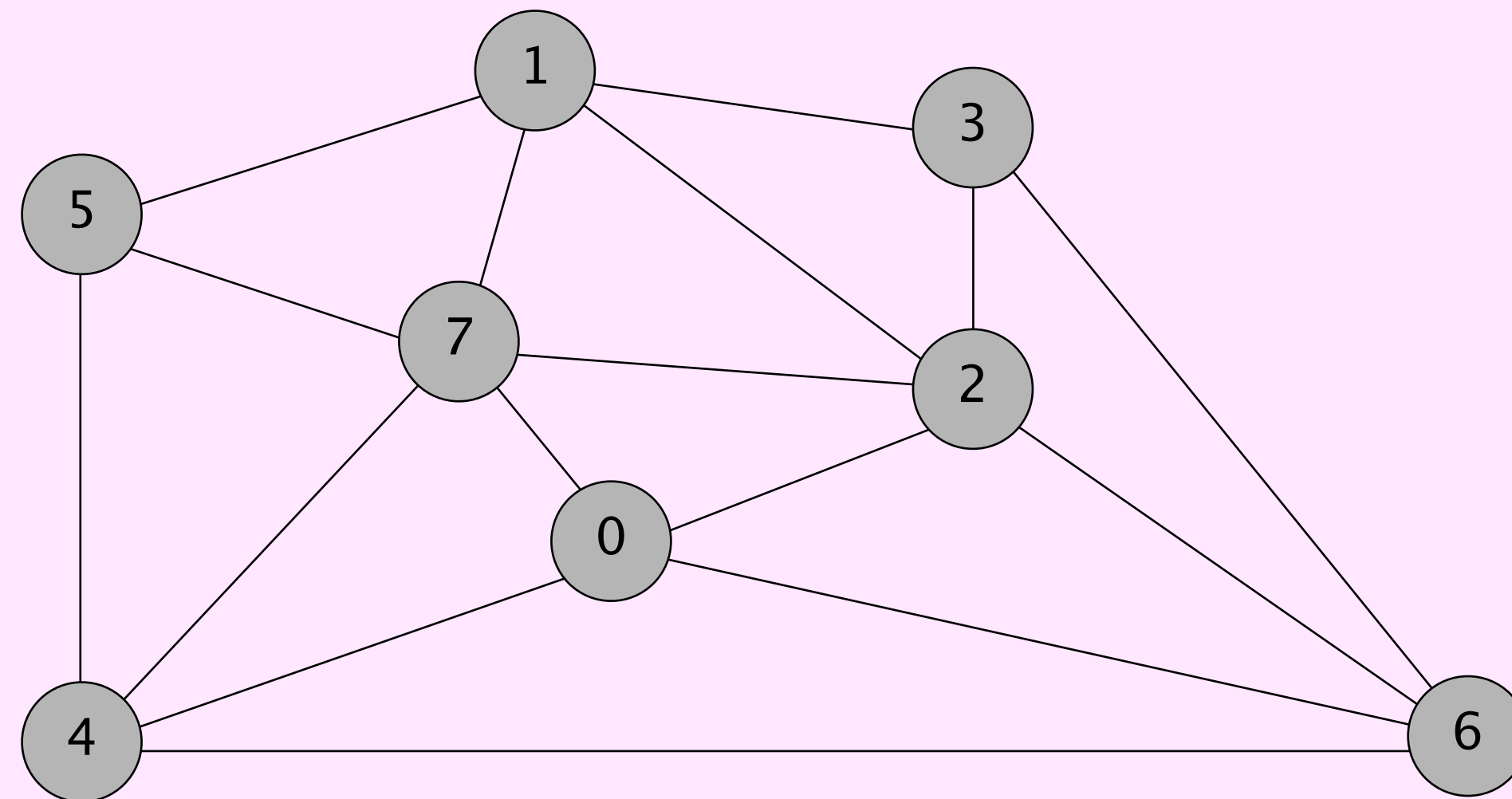
Challenge. How to efficiently find min-weight edge with exactly one endpoint in T ?



Prim's algorithm: lazy implementation demo



- Start with vertex 0 and grow tree T .
- Repeat until $V - 1$ edges:
 - add to T the min-weight edge with exactly one endpoint in T



an edge-weighted graph

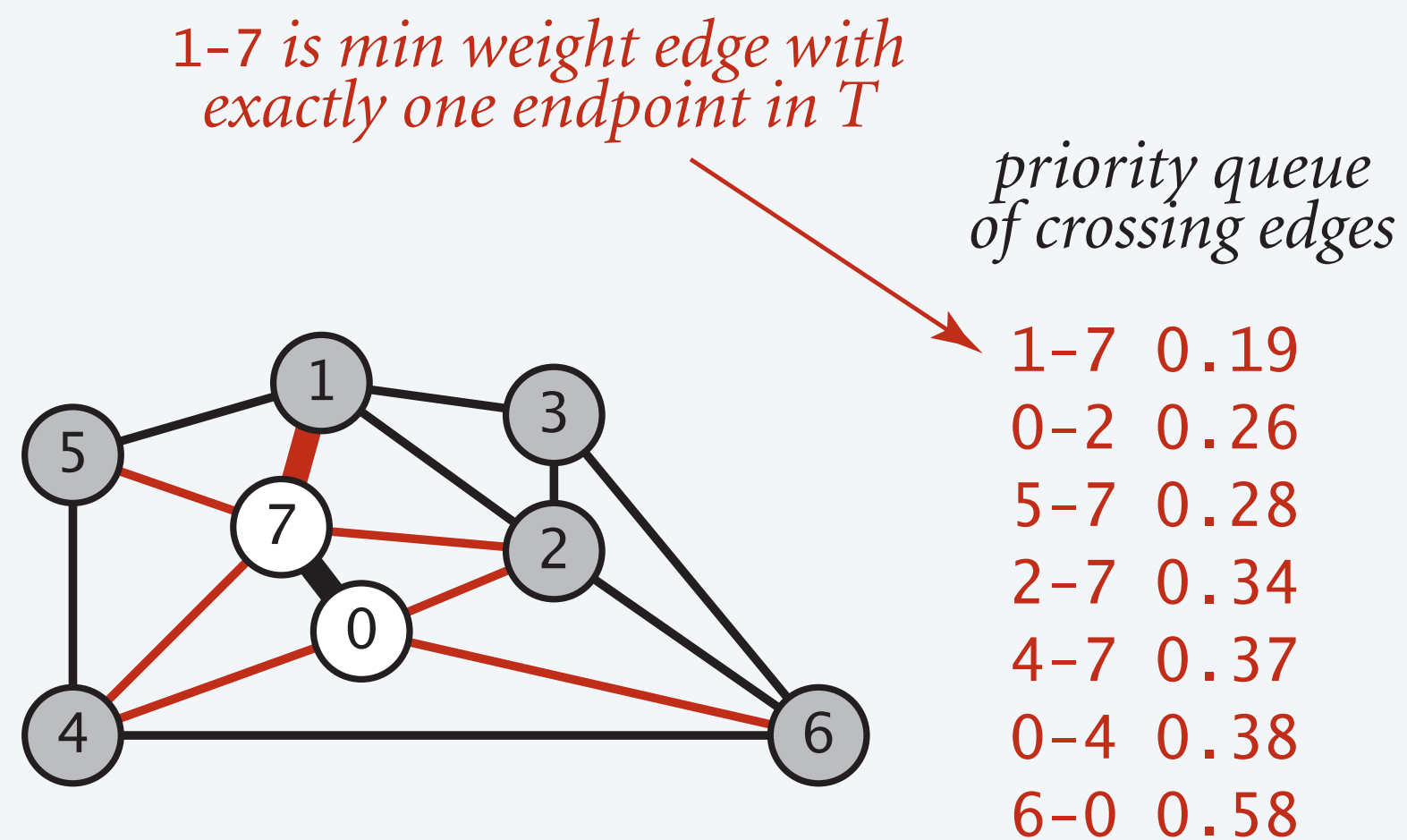
0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93

Prim's algorithm: lazy implementation

Challenge. How to efficiently find min-weight edge with exactly one endpoint in T ?

Lazy solution. Maintain a PQ of **edges** with (at least) one endpoint in T .

- Key = edge; priority = weight of edge.
- DELETE-MIN to determine next edge $e = v-w$ to add to T .
- If both endpoints v and w are marked (both in T), disregard.
- Otherwise, let w be the unmarked vertex (not in T):
 - add e to T and mark w
 - add to PQ any edge incident to w $\longleftarrow$ *but don't bother if other endpoint is already in T*



Prim's algorithm: lazy implementation

lazy: 不主动删除优先队列中已经无效的边 (两端都在 T 中的边), 而是在取出边时才判断是否有效, 无效则跳过, 属于「延迟处理」

```
public class LazyPrimMST {
    private boolean[] marked; // MST vertices
    private Queue<Edge> mst; // MST edges
    private MinPQ<Edge> pq; // PQ of edges

    public LazyPrimMST(WeightedGraph G) {
        pq = new MinPQ<>();
        mst = new Queue<>();
        marked = new boolean[G.V()];
        visit(G, 0); // assume graph G is connected
    }
    ...
}
```

```
while (mst.size() < G.V() - 1) {
    Edge e = pq.delMin(); 参考堆
    int v = e.either(), w = e.other(v);
    if (marked[v] && marked[w]) continue;
    mst.enqueue(e);
    if (!marked[v]) visit(G, v);
    if (!marked[w]) visit(G, w);
}
```

repeatedly delete the min-weight
edge $e = v-w$ from PQ

ignore if both endpoints in tree T 如果两个节点都已经在 T 中, 就ignore

add edge e to tree T

add either v or w to tree T

visit的除了操作节点, 还将所有相邻的边加入优先队列
也就是说, 优先队列中的所有边, 都至少有一个节点在
mst中

```
private void visit(WeightedGraph G, int v) {
    marked[v] = true; // add v to tree T
    for (Edge e : G.adj(v))
        if (!marked[e.other(v)])
            pq.insert(e);
}
```

```
public Iterable<Edge> mst() {
    return mst;
}
```

for each edge $e = v-w$:
add e to PQ if w not already in T

Lazy Prim's algorithm: running time

Proposition. In the worst case, lazy Prim's algorithm computes the MST in $\Theta(E \log E)$ time and $\Theta(E)$ extra space.

Pf.

- 
- Bottlenecks are PQ operations.
 - Each edge is added to PQ at most once.
 - Each edge is deleted from PQ at most once.

频率是E，是因为最坏情况时每一条边都要处理

operation	frequency	time per op
INSERT	E	$\log E^\dagger$
DELETE-MIN	E	$\log E^\dagger$

† using binary heap

Prim's algorithm: eager implementation

复杂度分析：最坏情况，每条边都要处理（E），每次处理都要更新PQ（插入操作复杂度 $\log n$ ，这里 n 是图的节点数量 $=V$ ，所以Eager是 $E \log V$ 时间复杂度

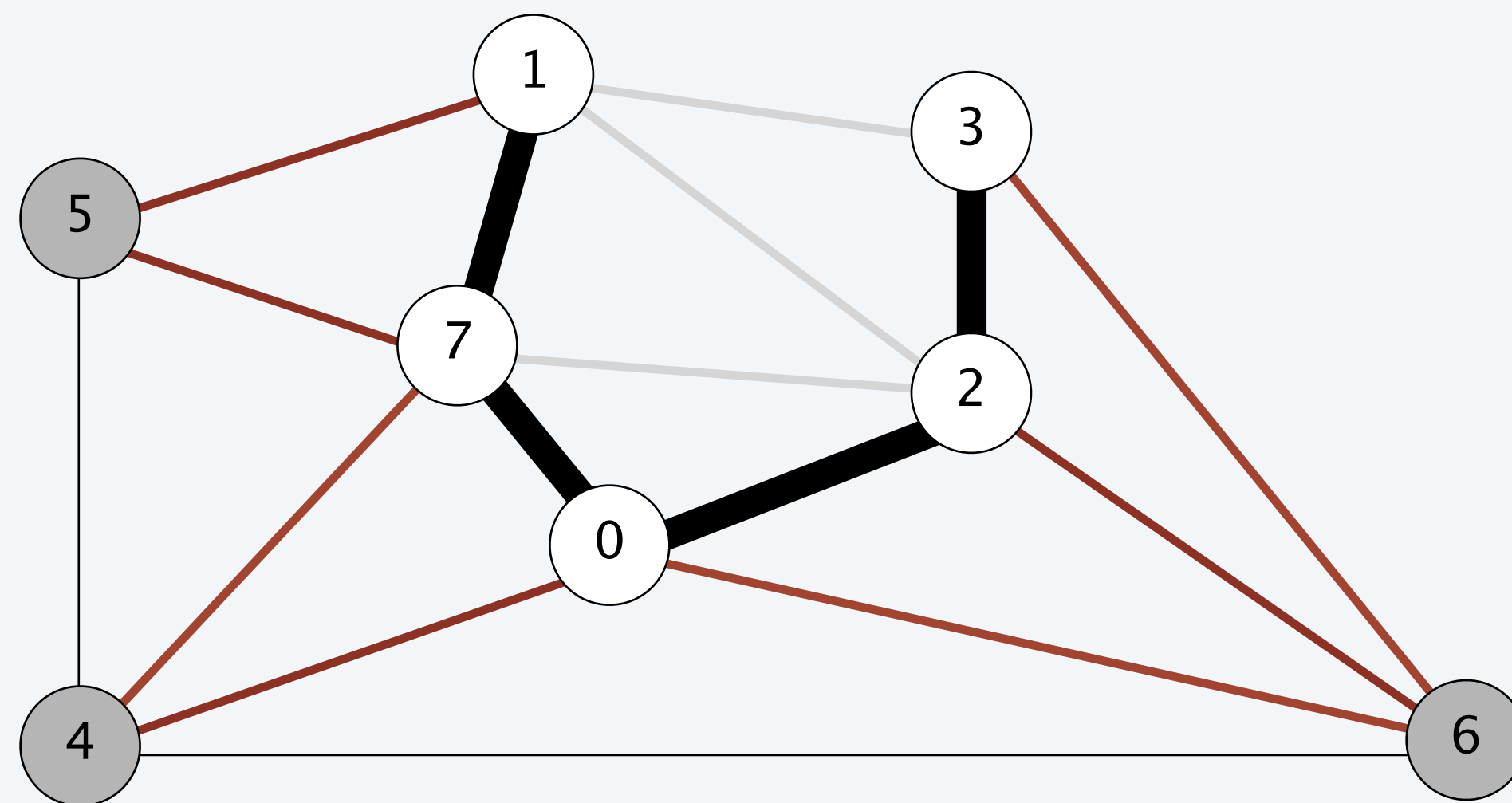
只存每个未加入顶点到 MST 的节点的最短距离（权重）

Challenge. Find min-weight edge with exactly one endpoint in T .

Observation. For each vertex v , need only **min-weight** edge connecting v to T .

- MST includes at most one edge connecting v to T . Why?
- If MST includes such an edge, it must take lightest such edge. Why?

Impact. PQ of **vertices**; $\Theta(V)$ extra space; $\Theta(E \log V)$ running time in worst case.



在visit的时候就维护距离当前处理节点的
最短的边和对应的节点

```
private void visit(WeightedGraph G, int v) {
    marked[v] = true;
    for (Edge e : G.adj(v)) {
        int w = e.other(v);
        if (!marked[w]) { // 这里就保证了w不在MST里面
            // 关键区别：这里不直接塞边，而是比较权重
            if (e.weight() < distTo[w]) {
                distTo[w] = e.weight(); // 更新距离
                edgeTo[w] = v;           // 更新父节点

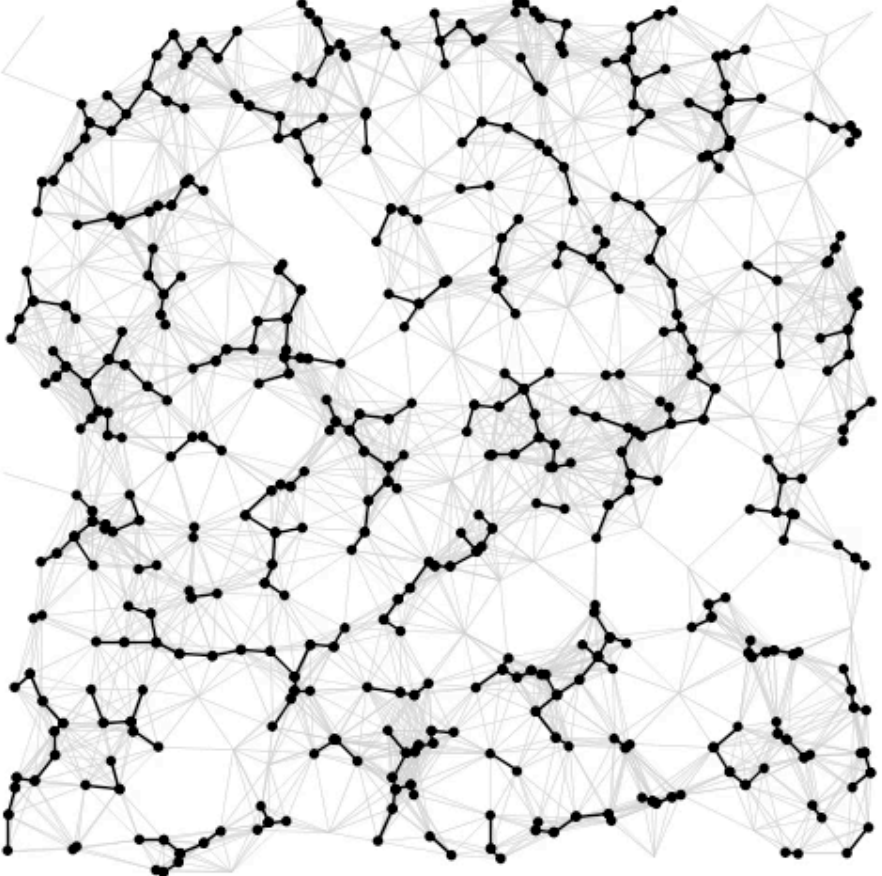
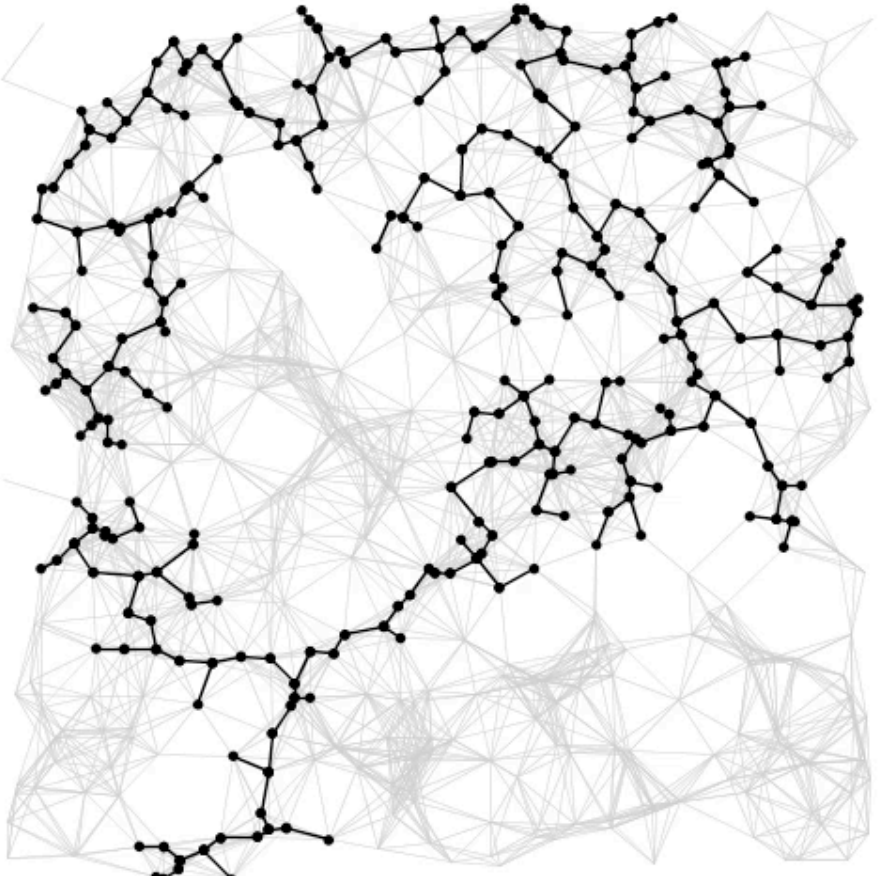
                if (pq.contains(w)) {
                    pq.decreaseKey(w, distTo[w]);
                    // 减小w权值（更新在pq中位置）
                } else {
                    pq.insert(w, distTo[w]); // 插入
                }
            }
        }
    }
}
```

see textbook
for details

```
// while 循环
while (mst.size() < G.V() - 1) {
    int v = pq.delMin(); // 取出权值最小的那个顶点
    mst.enqueue(edgeTo[v]); // 直接把对应的边加入MST

    visit(G, v); // 扩展这个顶点
}
```

MST: algorithms of the day

algorithm	visualization	bottleneck	running time
Kruskal		<i>sorting</i> <i>union-find</i>	$E \log E$ 和Prim的Lazy算法一样
Prim		<i>priority queue</i>	$E \log V$ 这是Eager算法的

Credits

image	source	license
<i>Muddy City Problem</i>	<u>CS Unplugged</u>	<u>CC BY-NC-SA 4.0</u>
<i>Microarrays and Clustering</i>	Botstein and Brown	by author
<i>Image Segmentation</i>	<u>Felzenszwalb and Huttenlocher</u>	
<i>Phylogeny Tree</i>	<u>Derzelle et al.</u>	
<i>MST Dithering</i>	<u>Mario Klingemann</u>	<u>CC BY-NC 2.0</u>
<i>Slime Mold vs. Rail Network</i>	<u>Harvard Magazine</u>	
<i>Mona Singh</i>	<u>Princeton University</u>	

A final thought

“ The algorithms we write are only as good as the questions we ask. And the best questions come from collaboration and creative thinking. ” — [Mona Singh](#)

